Bài 1 Tạo Smartcontract với Solidity

Trong bài này sẽ hướng dẫn cách tạo 1 Smart Contract ERC20 USDT, biên dịch và deploy nó như thế nào.

1. Cài đặt Metamask extension trên trình duyệt

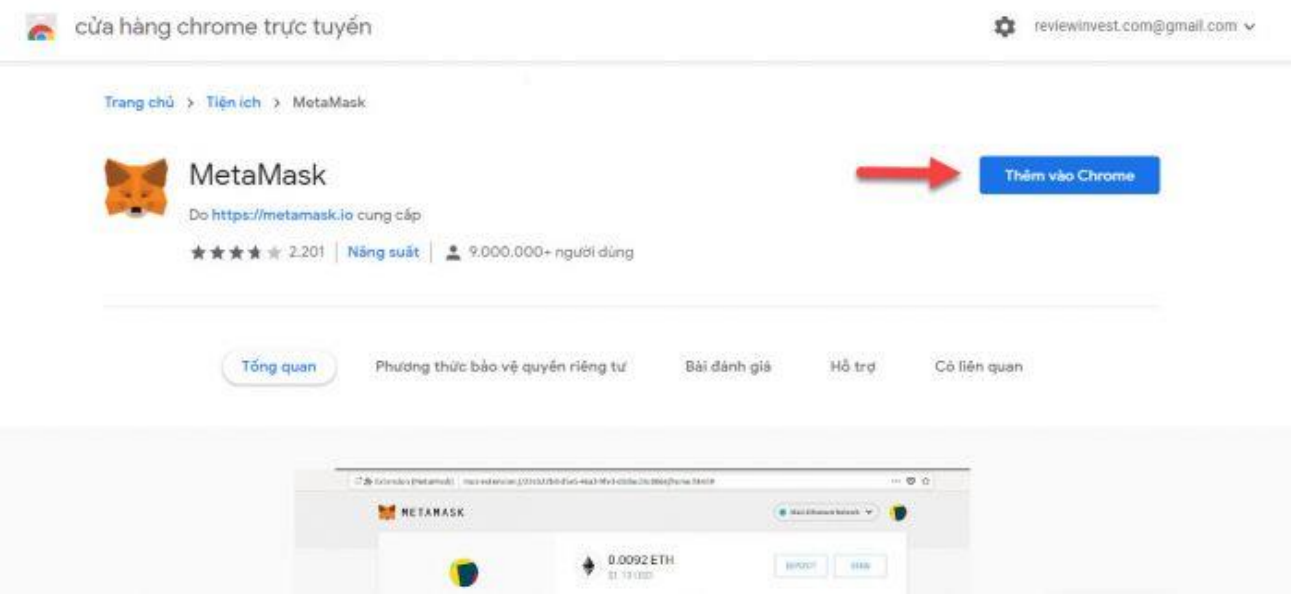
Hướng dẫn này sẽ hướng dẫn thao tác cài đặt ví Metamask trên Chrome một cách đơn giản, tiện lợi và dễ dàng sử dụng cho tất cả thiết bị Chrome trên máy tính.

a. Thêm tiện ích Metamask vào trình duyệt

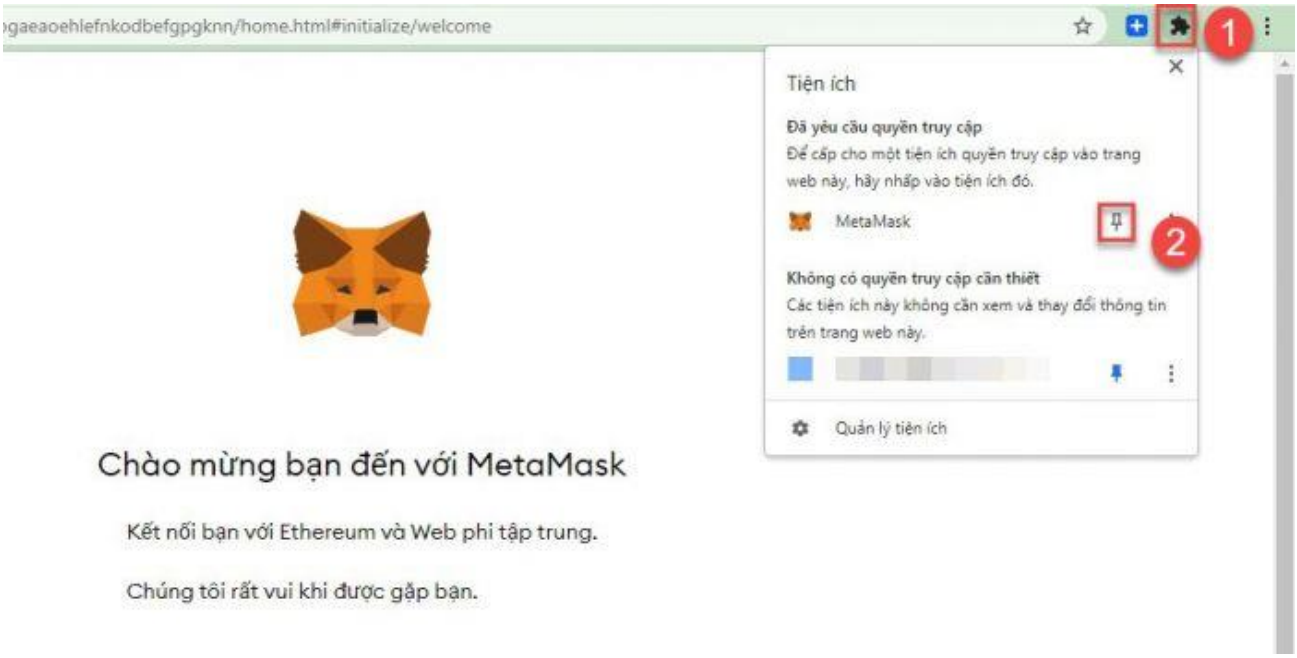
Bước 1: Thêm tiện ích Metamask trên trình duyệt

- Bạn vào Metamask.io để cài Metamask, thêm vào Chrome hoặc truy cập trực tiếp link sau để cài tiện ích Metamask trên Chrome:

https://chromewebstore.google.com/detail/metamask/nkbihfbeogaeaoehlefnkodbefgpgknn



- Sau khi nhấn "**Thêm tiện ích**", bạn sẽ mở thanh tiện ích như ảnh để mở ví. Bạn có thể nhấn biểu tưởng "**Ghim**" để biểu tượng ví luôn hiển thị sẵn.

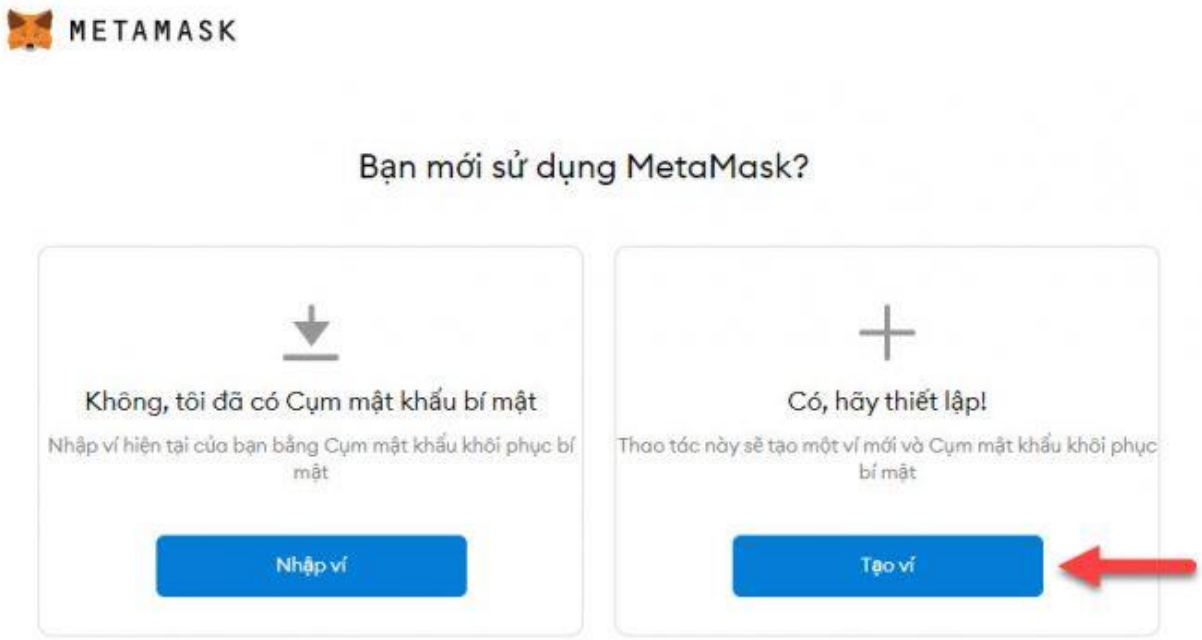


- Sau đó, bạn nhấn vào "Bắt đầu" để sang bước tiếp theo -Tạo/đăng nhập ví.

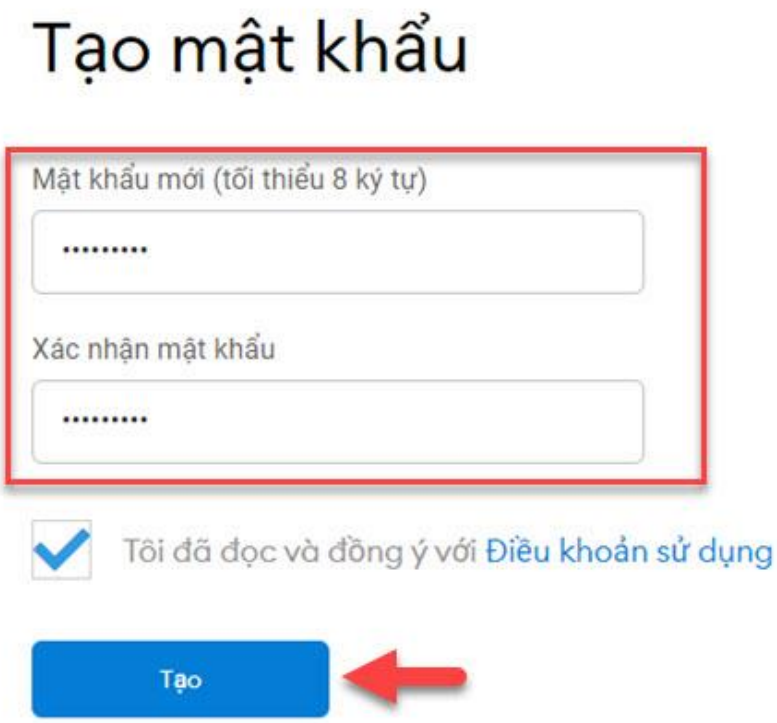
Bước 2: Tạo hoặc đăng nhập ví Metamask

Sẽ có 2 lựa chọn:

- **Không, tôi đã có Cụm mật khẩu bí mật**: Bạn sẽ đăng nhập vào ví bằng 12 ký tự bí mật, áp dụng với người đã tạo ví trước đó.
- **Có, hãy thiết lập!:** Tạo ví mới.



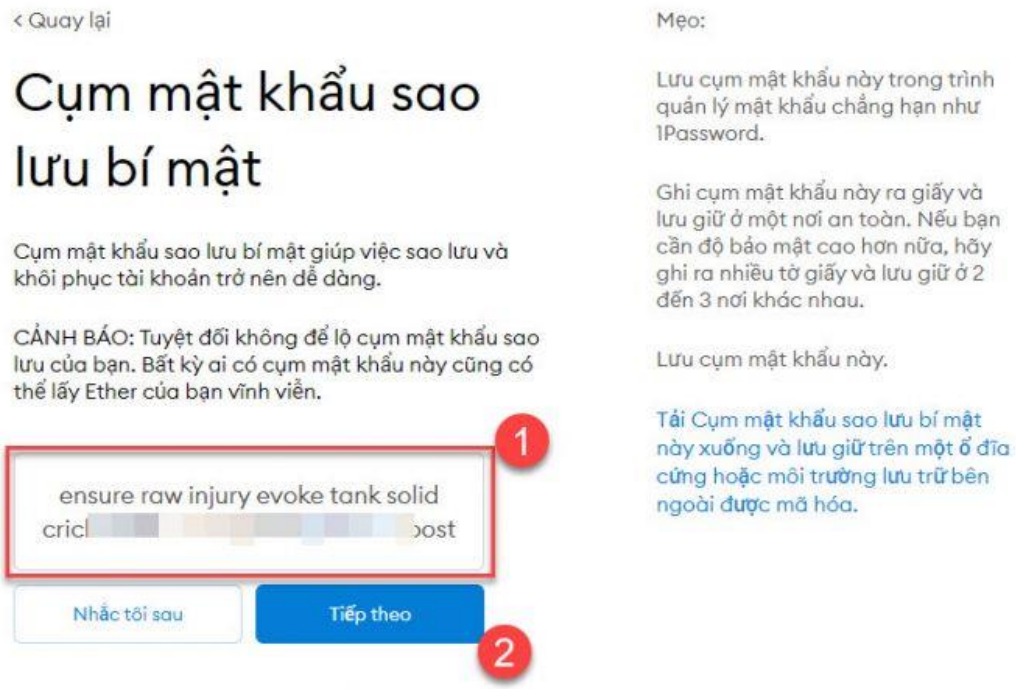
- Sau khi đọc các điều khoản hiện ra, bạn nhấn "Tôi đồng ý" để tạo mật khẩu.



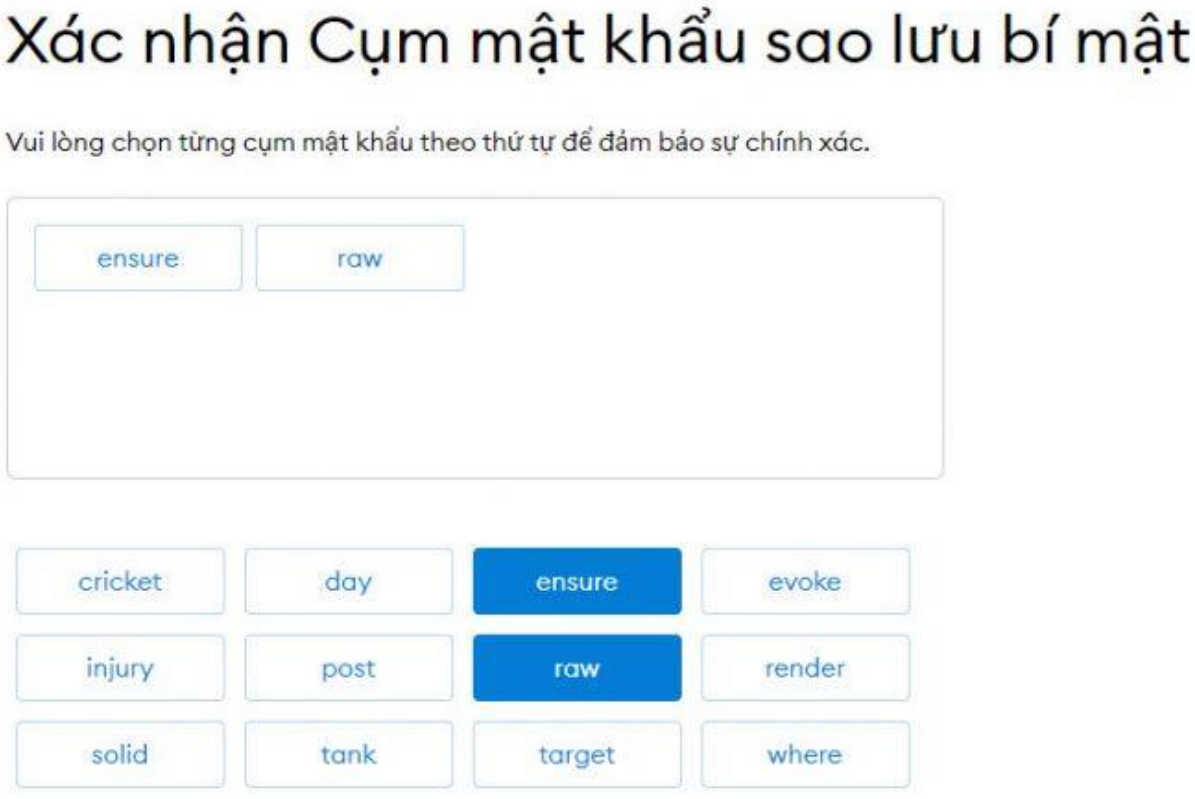
Metamask sẽ đưa cho bạn 12 từ khóa gọi là **Secret Backup Phrase** (1). Sau khi lưu lại bằng cách viết ra hoặc chụp ảnh cất giữ, bạn nhấn "**Tiếp theo**" (2).

Lưu ý:

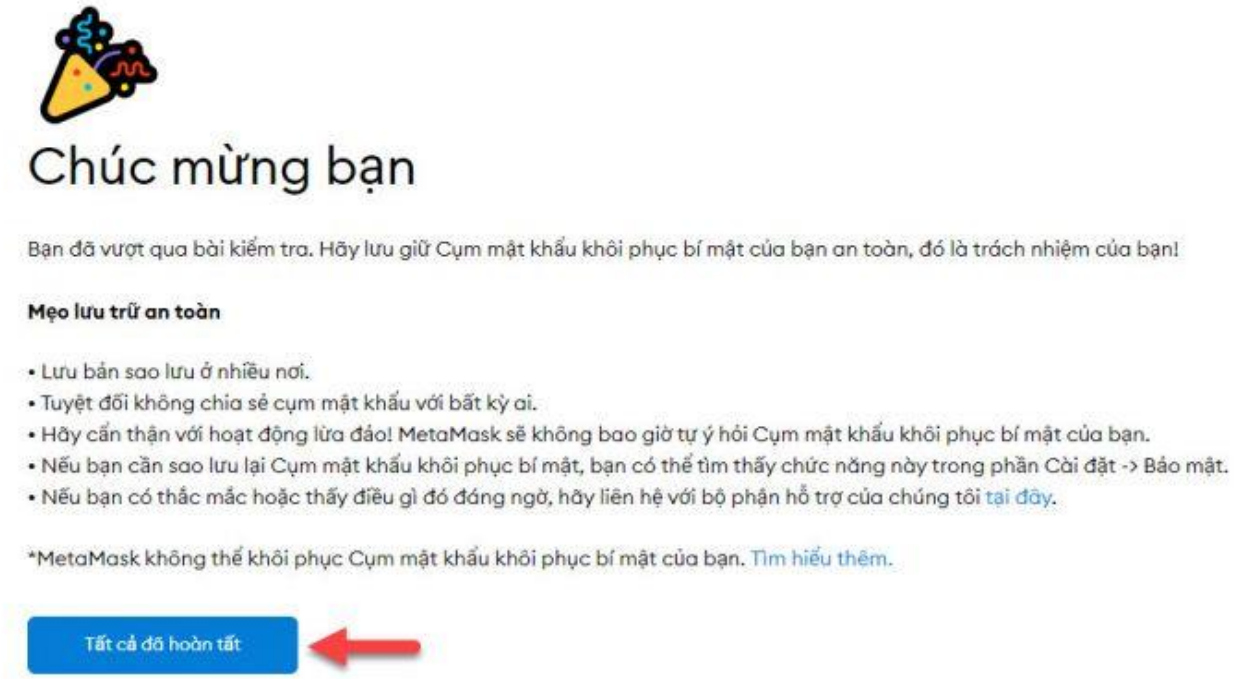
- Secret Backup Phrase này quan trọng nhất, hơn cả mật khẩu bạn tạo ở trên nữa. Bạn cần phải nhập 12 từ khóa này để khôi phục và đổi lai mật khẩu. Nên nếu mất hoặc bị lộ 12 từ khóa này là coi như mất luôn ví.
- 12 từ khóa này không thay đổi hay lấy lại được nếu bị mất.
- Do đó, bạn nên lưu và bảo mật nó thật kỹ, có thể viết ra giấy cho chắc chắn! Và tuyệt đối không đưa Secret Backup Phrase cho bất kỳ ai!



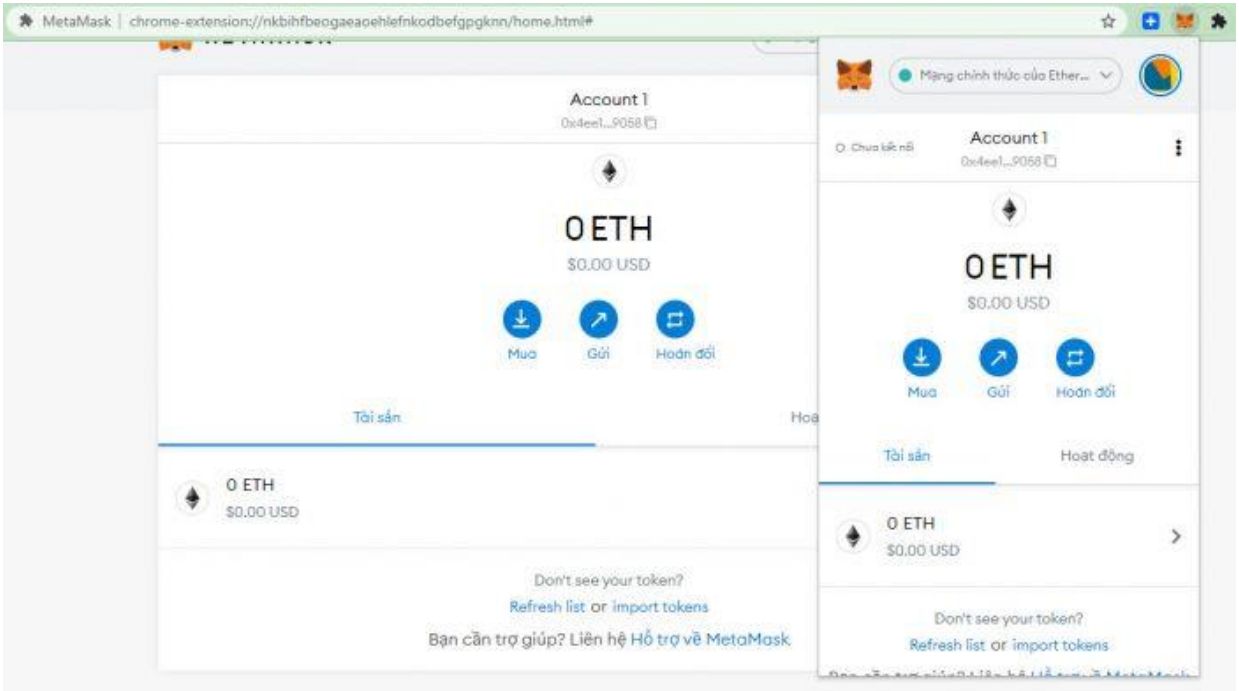
- Tiếp theo bạn sẽ cần nhập lại 12 từ khóa trước theo thứ tự chính xác.



- Sau khi nhập xong, bạn nhấn "Xác nhận" để hoàn tất tạo ví.



- Vậy là bạn đã Hoàn tất việc Tạo ví Metamask rồi đó ạ! Đây là giao diện trên web và tiện ích (ảnh nhỏ góc phải ở trên)



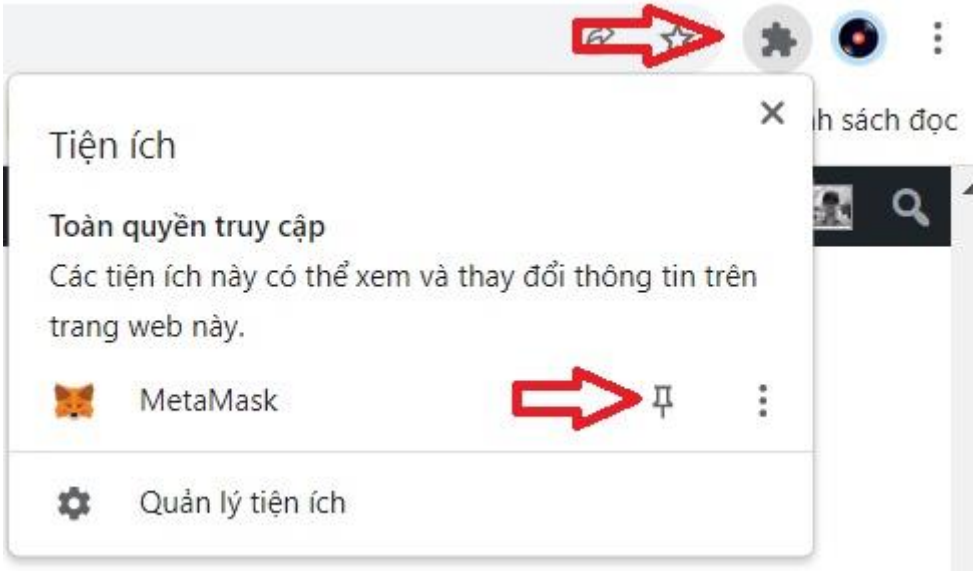
Bước 3: Đăng nhập ví Metamask

- Mỗi lần bạn tắt trình duyệt web, để mở lại ví bạn cần nhập lại bằng mật khẩu. Tuy nhiên, nếu lỡ quên, bạn có thể dùng 12 từ khóa bí mật để khôi phục lại ví và tạo mật khẩu mới. Vì thế mà 12 từ khóa này cực kỳ quan trọng là vậy.

Ghim Metamask để tiện sử dụng

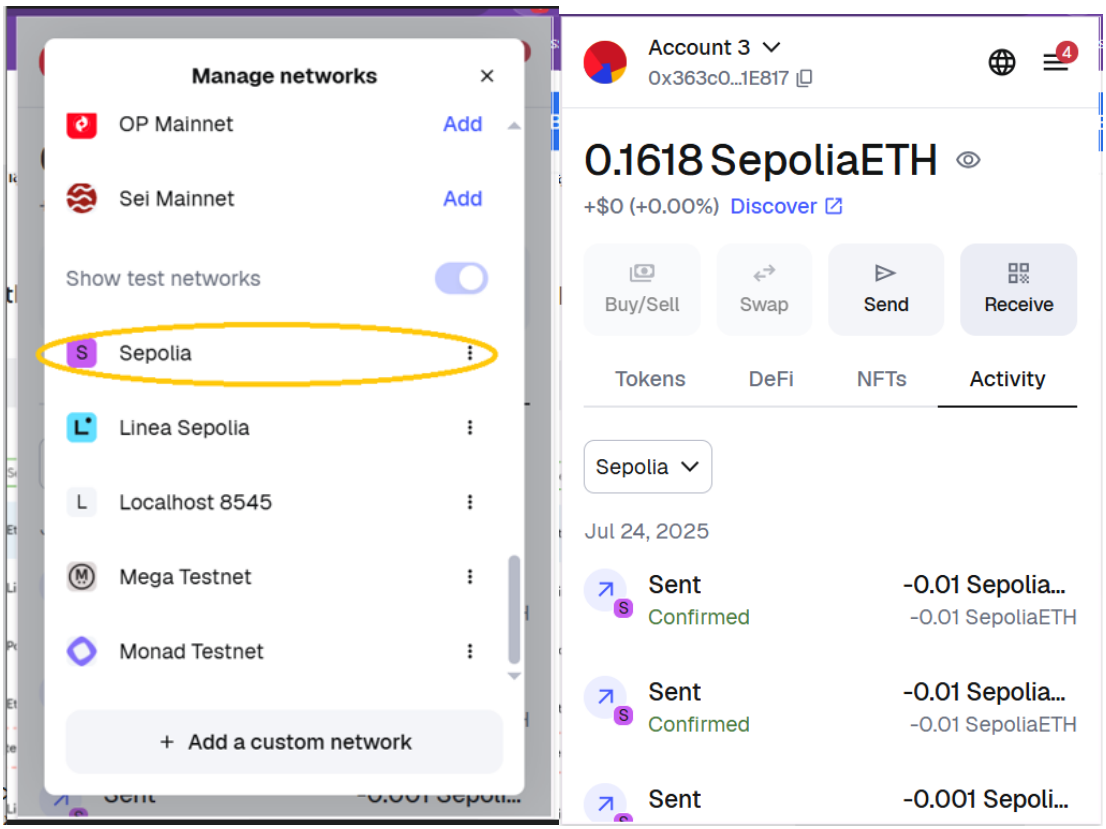
- Kích vào biểu tượng quản lý tiện ích góc trên bên phải Chrome.
- Tích ghim Metamask để tiện sử dụng.

Khi bật/tắt Chrome sẽ yêu cầu bạn nhập mật khẩu để truy cập ví Metamask.



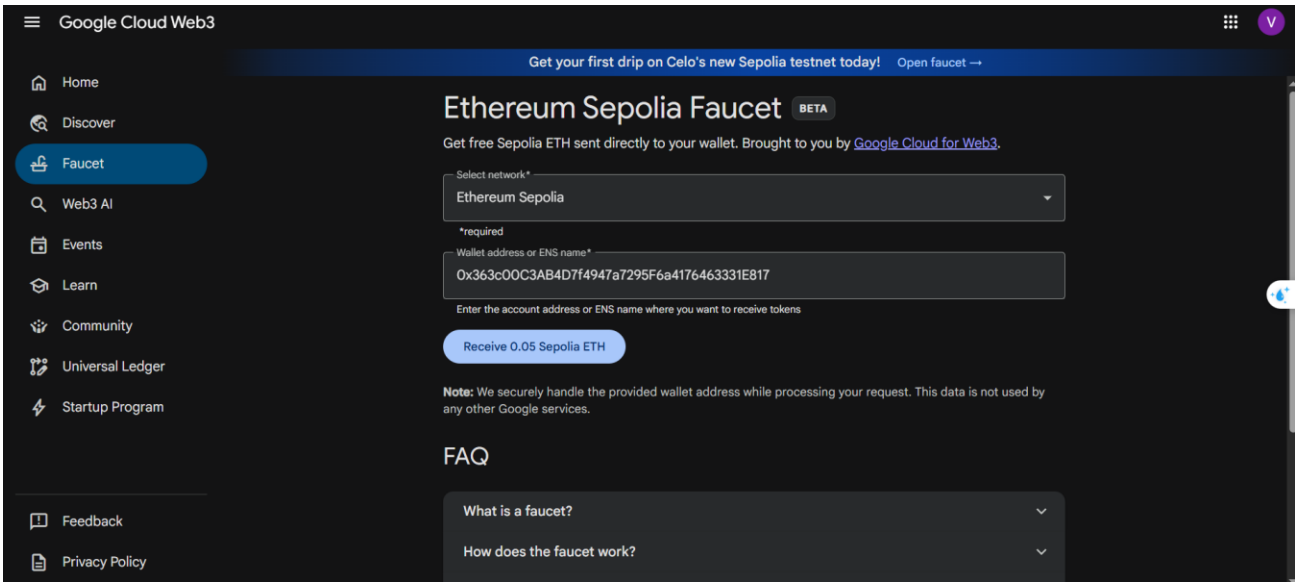
b. Chọn mạng lưới Testnet (Sepolia chain) vào Metamask

1. Đăng nhập vào tiện ích Metamask
2. Chọn biểu tượng Menu ở góc trên bên phải của tiện ích Metamask
3. Chọn Networks
4. Kéo xuống dưới và chọn “Sepolia”



c. Hướng dẫn Faucet amount mạng Sepolia testnet

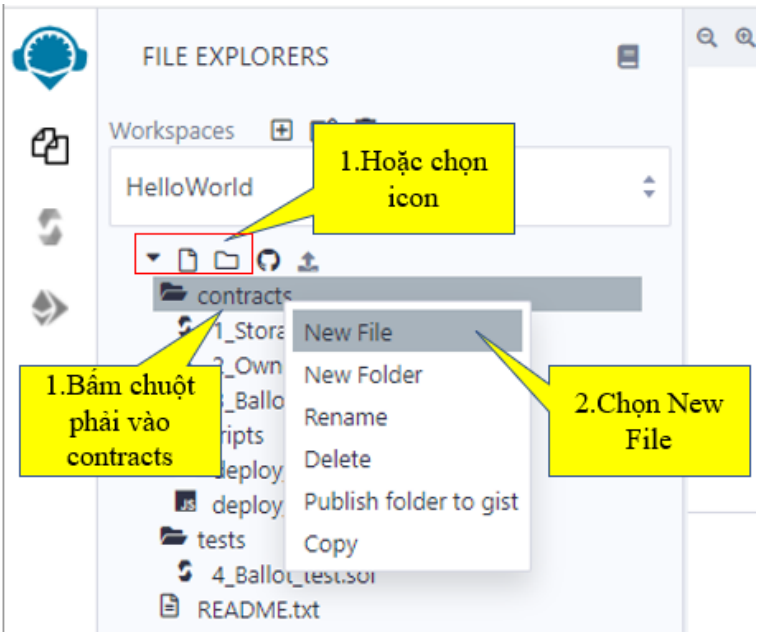
1. Truy cập đường link để sau: https://cloud.google.com/application/web3/faucet/ethereum/sepolia
2. Trong “Select network” chọn “Ethereum Sepolia”
3. Sao chép địa chỉ ví ở Metamask và dán vào “Wallet address or ENS name”
4. Ấn “Receive 0.05 Sepolia ETH”



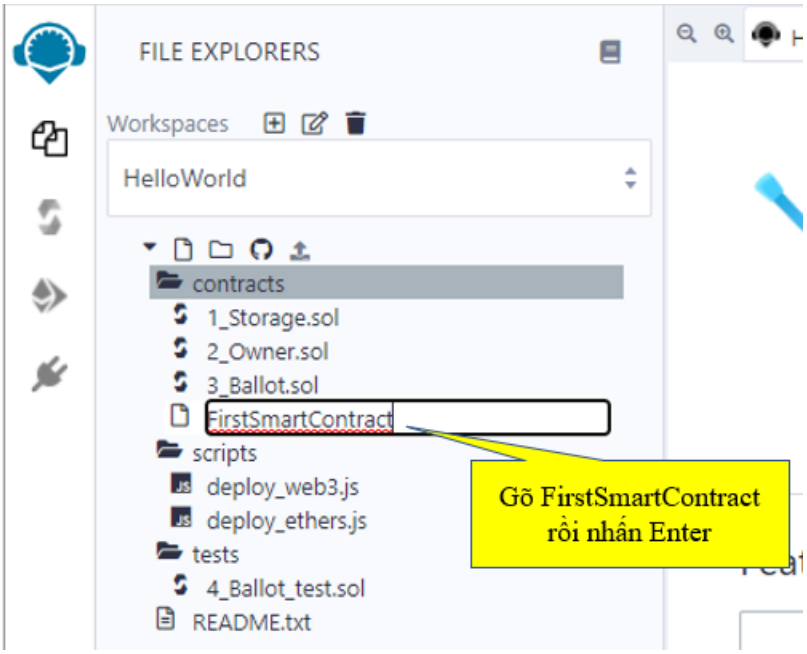
2. Sử dụng công cụ Remix IDE để tạo Smartcontract

Link truy cập: https://remix.ethereum.org/

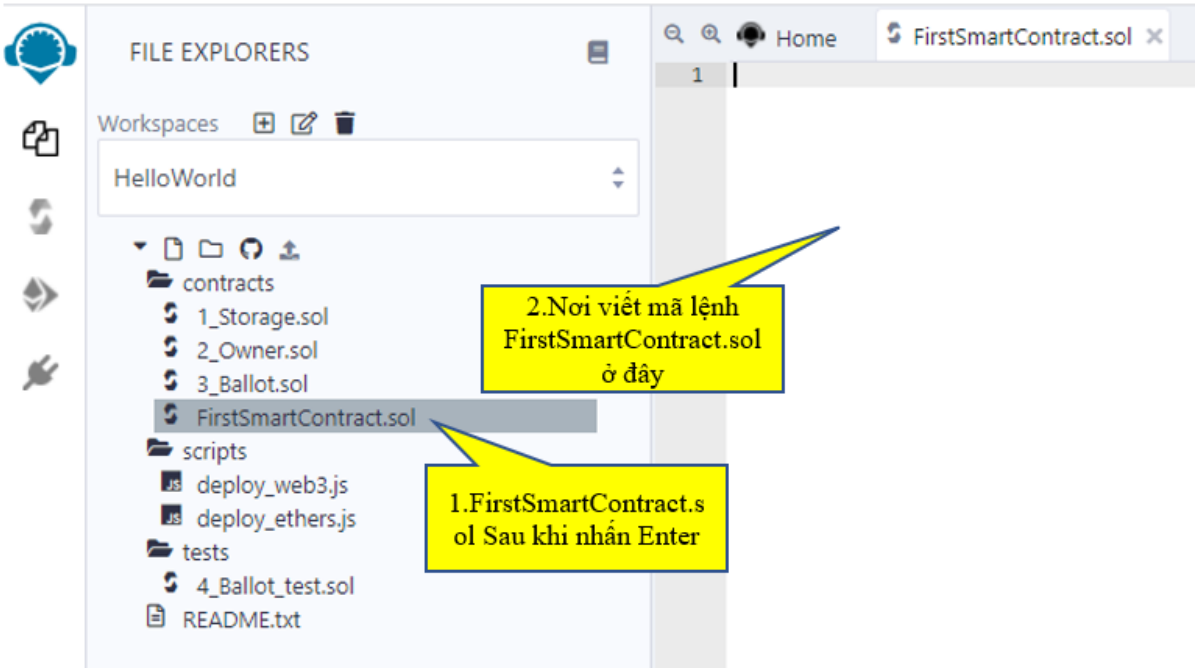
- Đầu tiên tạo 1 file .sol trong "contracts"



- Đặt tên cho file .sol đó là FirstSmartContract, nó tự ra đuôi FirstSmartContract.sol:

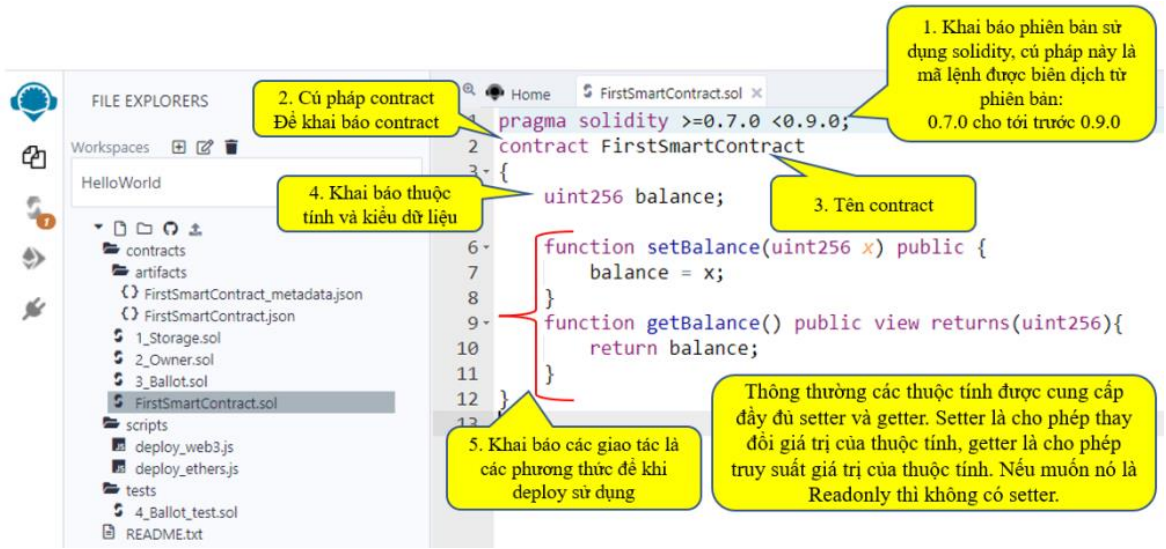


- Kết quả sau khi nhấn phím Enter:



- Bây giờ ta code như sau, cách code Solidity nó tương tự như java, python…. dĩ nhiên có những điểm khác vì nó chuyên biệt cho Smartcontract.

Ví dụ:

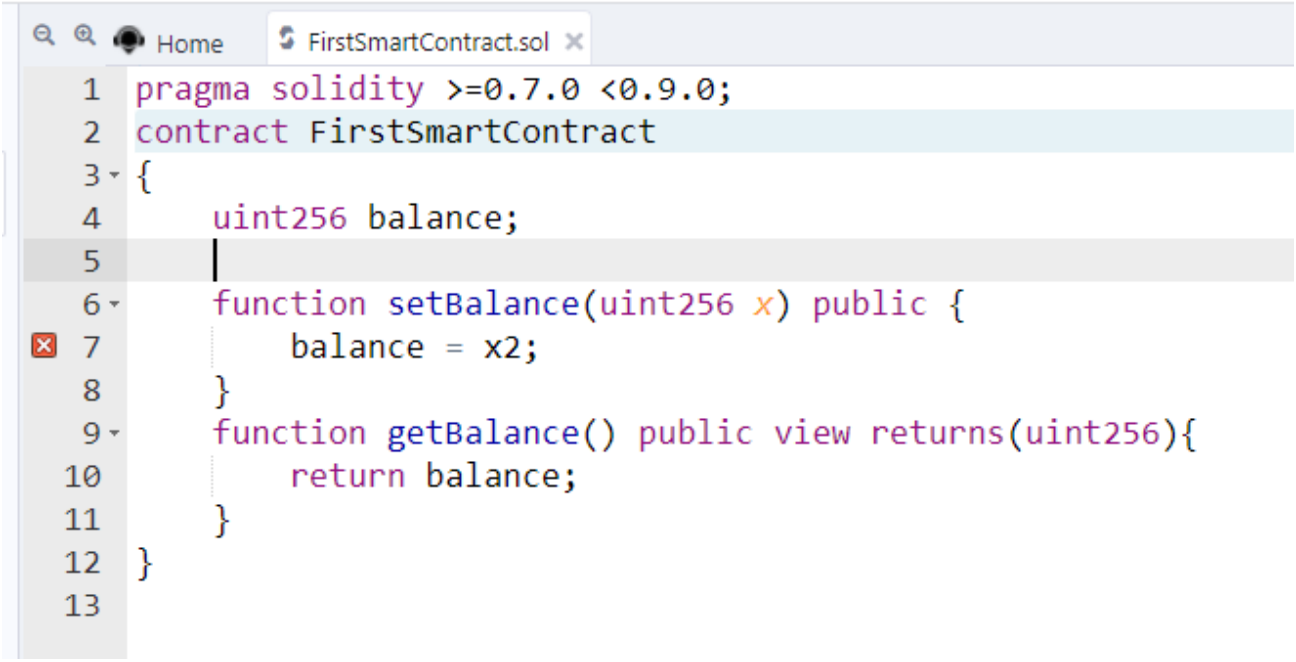


Coding như sau:

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.7.0 <0.9.0;
contract FirstSmartContract
{
    uint256 balance;
```

```
    function setBalance(uint256 x) public {
        balance = x;
    }
    function getBalance() public view returns(uint256){
        return balance;
    }
}
```

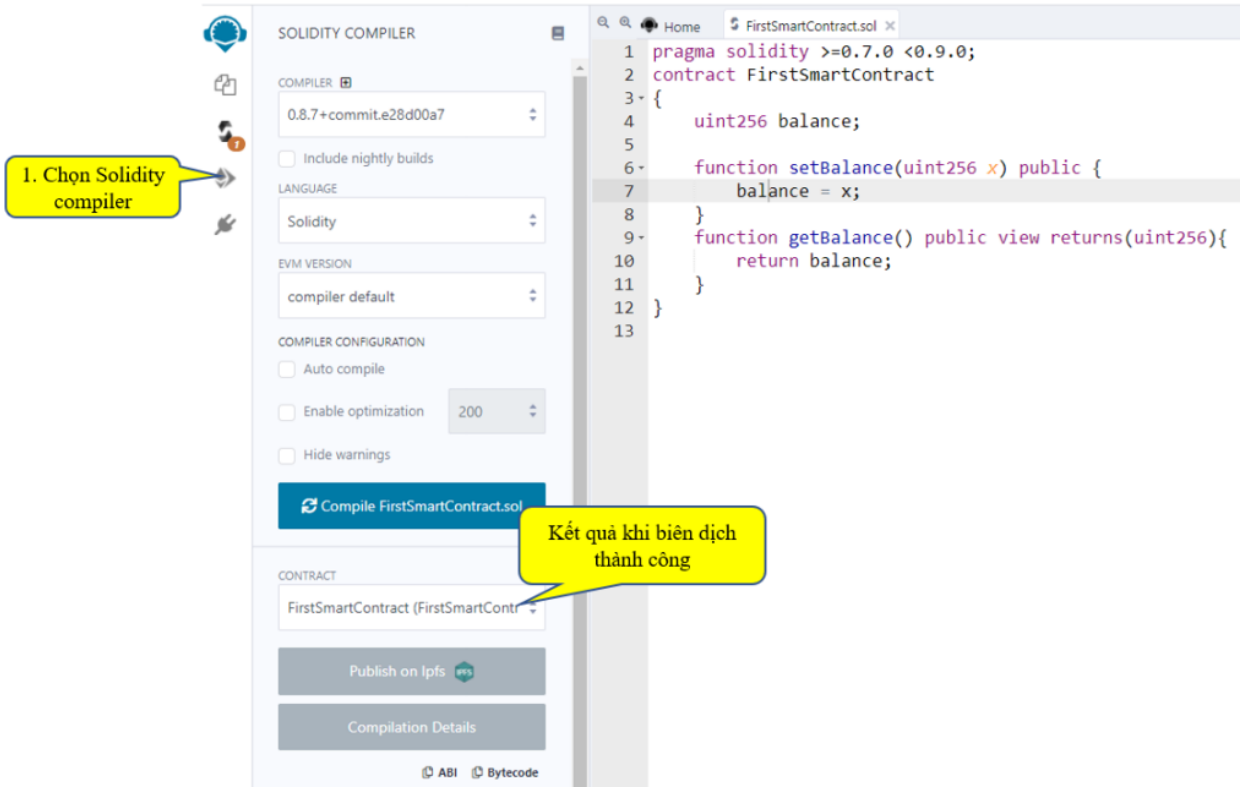
- Khi ta viết sai cú pháp thì chương trình sẽ tự động báo lỗi, lúc này ta sẽ không biết dịch được, ví dụ nó báo lỗi thì bạn sẽ thấy hình ảnh sau:



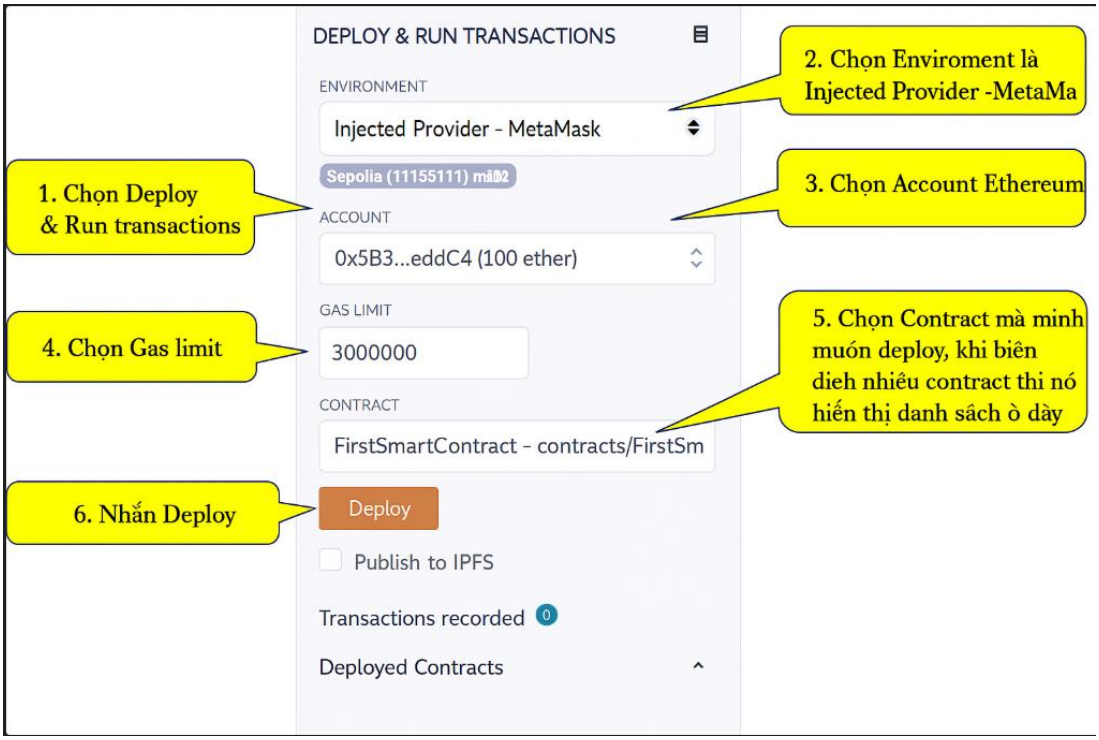
- Theo hình trên, để biên dịch ta làm theo cá bước:

1. Chọn Solidity compiler
2. Chọn compiler mới nhất
3. Chọn ngôn ngữ lập trình là Solidity
4. Chọn Ethereum Virtual Machine (EVM), để mặc định
5. Nhấn compile FirstSmartContract.sol

- Ta xem kế quả:



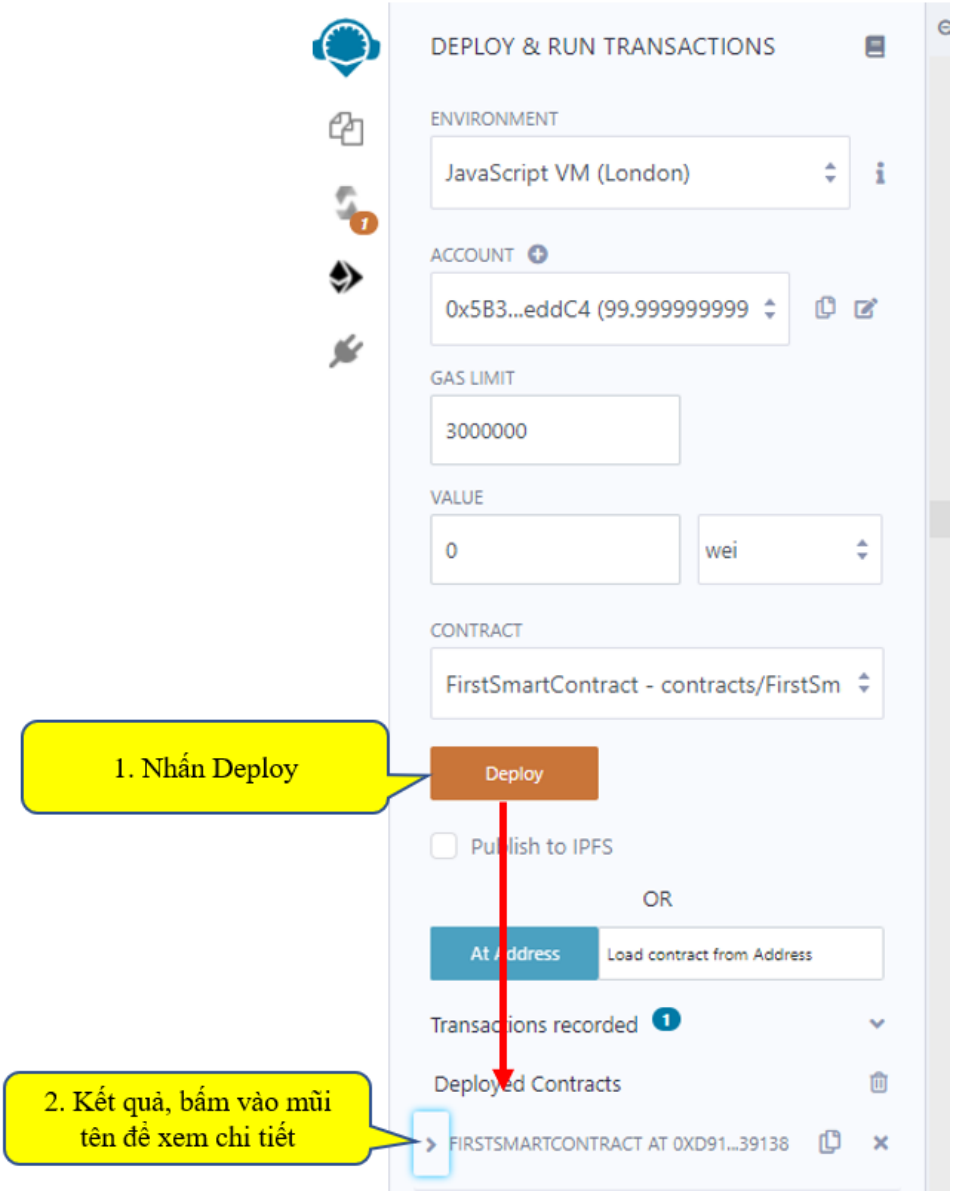
- Sau khi biên dịch thành công, ta vào mục Deploy & Run transactions như hình dưới đây:



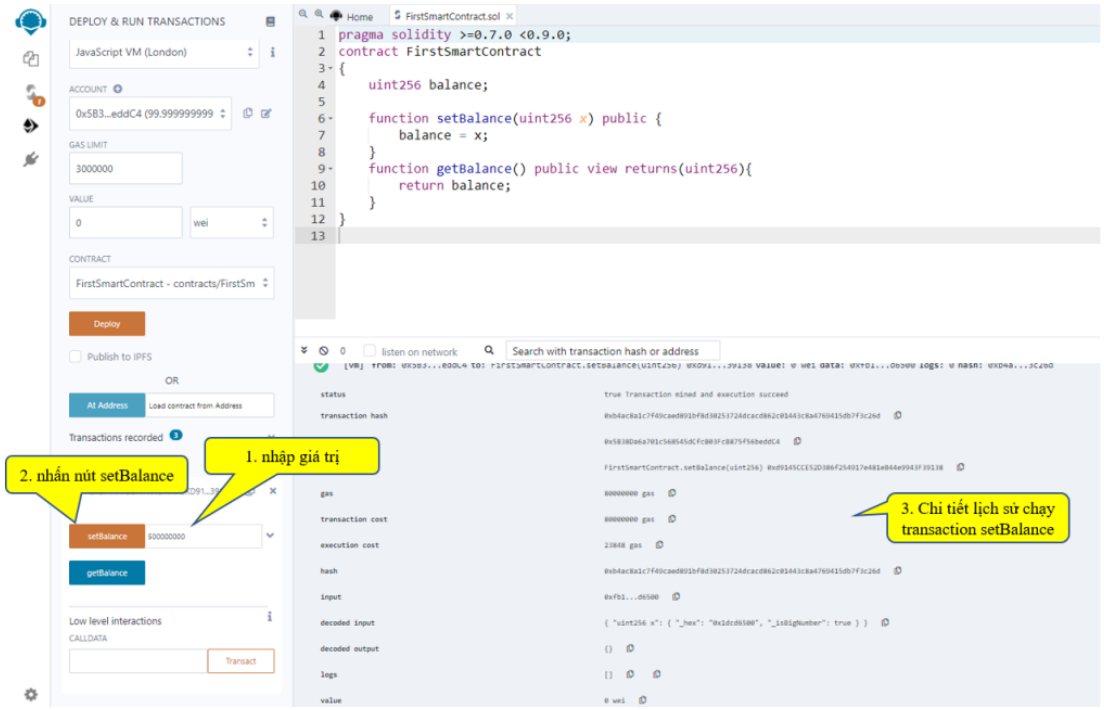
- Như vậy các bạn cần làm các bước trong mục Deploy & run transactions như sau:

1. Chọn “Deploy & Run transactions”
2. Chọn “Injected Provider – Metamask”
3. Chọn Account Ethereum
4. Chọn Gas limit
5. Chọn Contract mà mình muốn deploy, khi biên dịch nhiều contract thì nó hiển thị danh sách ở đây
6. Nhấn Deploy

- Sau khi nhấn Deploy ta xem kết quả:

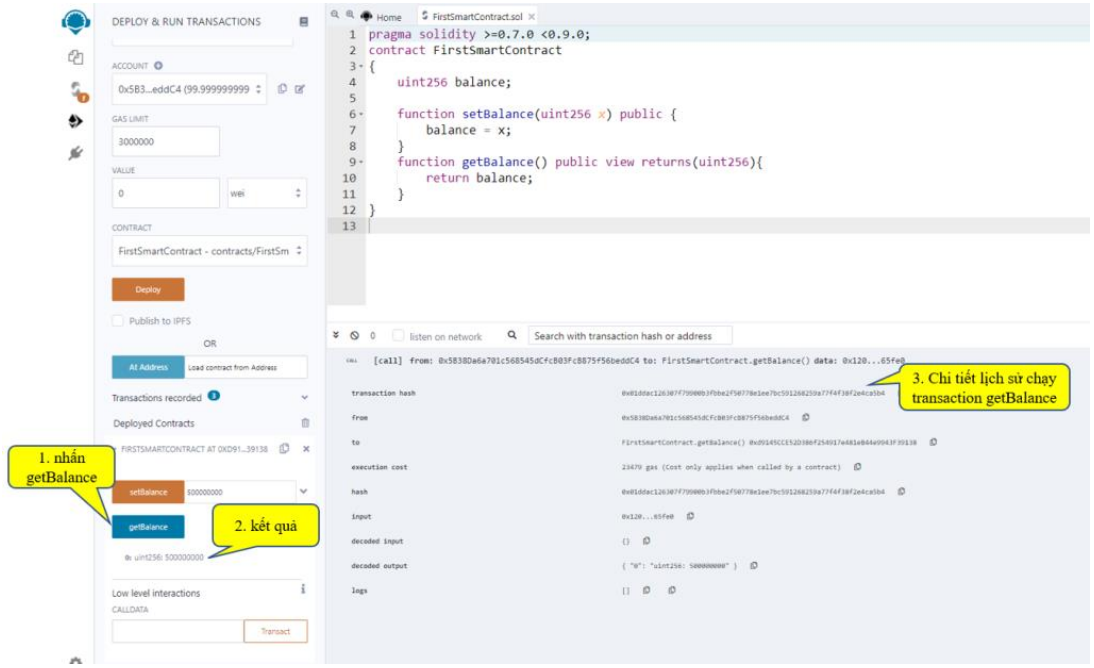


- Ví dụ bây giờ ta setBalance là 500000000, chi tiết thao tác xem hình:



- Bước 1: nhập giá trị cho giao tác setBalance
- Bước 2: nhấn nút setBalance
- Bước 3: chi tiết transaction thực hiện, ta có thể xem

- Tiếp theo để kiểm tra giao tác setBalance có làm thay đổi giá trị của balance hay không, ta thử nghiệm giao tác getBalance:



- Như vậy là tới đây ta đã hoàn tất việc Code một Smart Contract cũng như cách biên dịch và deploy, run nó như thế nào.

3. Sử dụng công cụ Hardhat để tạo Smartcontract

Yêu cầu nodejs version >= v16.xx

Bước 1. Tạo thư mục dự án mới

- mkdir smartcontract
- cd smartcontract

Bước 2. Khởi tạo dự án Node.js

- npm init -y

- Thao tác này sẽ tạo ra một "**package.json**" tệp quản lý các phụ thuộc của dự án bạn.

Bước 3. Cài đặt Hardhat

- Bây giờ, hãy cài đặt Hardhat cục bộ vào dự án của bạn:

- npm install --save-dev hardhat@^2.0.0
- npm i -D @nomiclabs/hardhat-waffle ethereum-waffle @nomiclabs/hardhat-ethers ethers chai

- Sau khi cài đặt, hãy tạo một dự án Hardhat mới:

- npx hardhat

- Bạn sẽ nhận được lời nhắc hỏi bạn muốn làm gì. Chọn:

"Tạo một dự án mẫu cơ bản" → Nhấn Enter → Nhập "có" khi được yêu cầu cài đặt các gói phụ thuộc.

```
D:\IT\SmartContract\smartcontract>npx hardhat
888    888                      888 888               888
888    888                      888 888               888
888    888                      888 888               888
8888888888  8888b.  888d888 .d88888 88888b.   8888b.  888888
888    888     "88b 888P"  d88" 888 888 "88b     "88b 888
888    888 .d888888 888    888  888 888  888 .d888888 888
888    888 888  888 888    Y88b 888 888  888 888  888 Y88b.
888    888 "Y888888 888     "Y88888 888  888 "Y888888  "Y888

Welcome to Hardhat v2.0.0

√ What do you want to do? · Create a sample project
√ Hardhat project root: · D:\IT\SmartContract\smartcontract
√ Do you want to add a .gitignore? (Y/n) · y

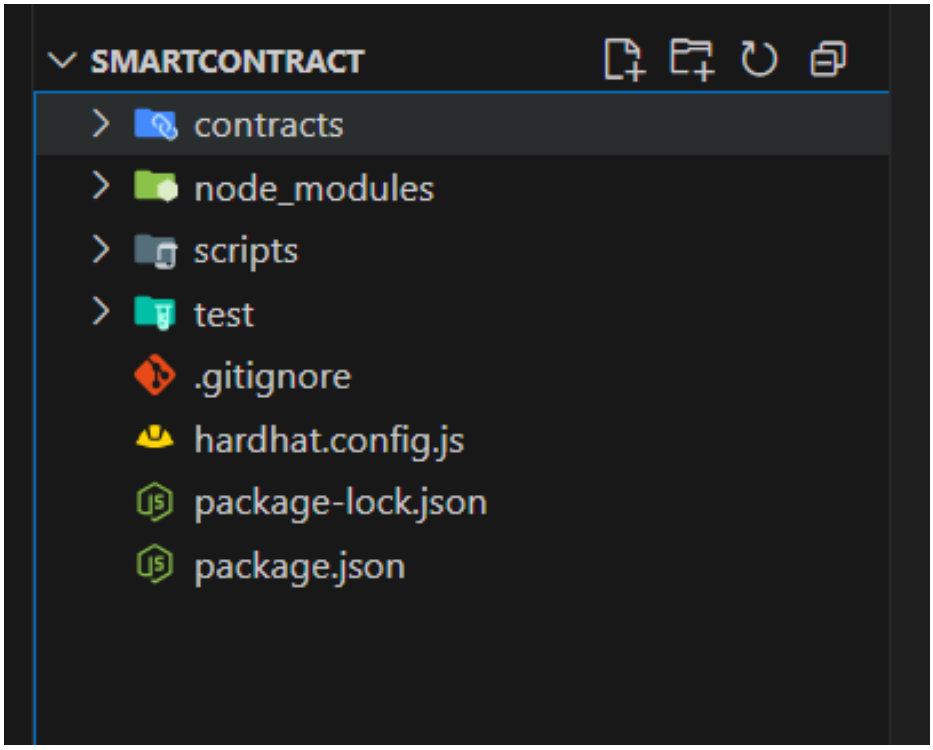
You need to install these dependencies to run the sample project:
  npm install --save-dev "hardhat@^2.0.0" "@nomiclabs/hardhat-waffle@^2.0.0" "ethe
reum-waffle@^3.0.0" "chai@^4.2.0" "@nomiclabs/hardhat-ethers@^2.0.0" "ethers@^5.0.
0"

Project created

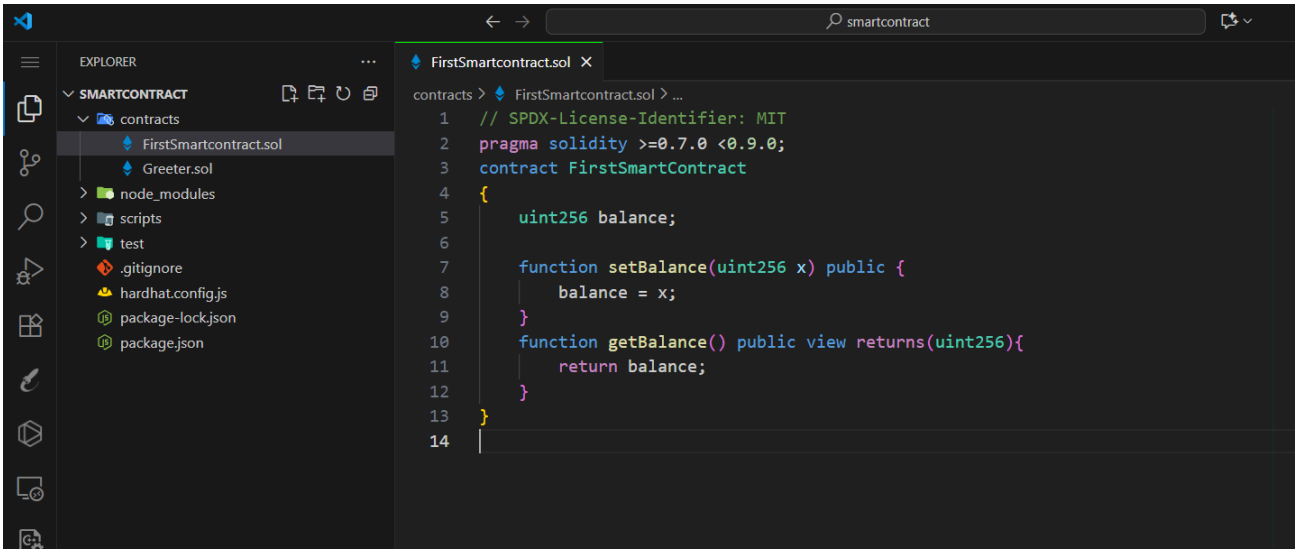
Try running some of the following tasks:
  npx hardhat accounts
  npx hardhat compile
  npx hardhat test
  npx hardhat node
  node scripts/sample-script.js
  npx hardhat help

D:\IT\SmartContract\smartcontract>
```

- Bây giờ bạn sẽ có cấu trúc thư mục như sau:



- Sau đó vào contracts thư mục và tạo một tệp mới có tên là FirstSmartcontract.sol và kết quả:



- Bây giờ, chúng ta hãy biên soạn hợp đồng:

- npx hardhat compile

- Nếu mọi việc diễn ra tốt đẹp, bạn sẽ thấy nội dung tương tự như sau: "Compilation finished successfully"

- Sau khi biên dịch, Hardhat tạo một thư mục mới có tên là **artifacts/**

- Tiến hành deploy smartcontract:

- Sửa file "hardhat.config.js" trong dự án:

```
require("@nomiclabs/hardhat-waffle");

// This is a sample Hardhat task. To learn how to create your own
go to
// https://hardhat.org/guides/create-task.html
task("accounts", "Prints the list of accounts", async () => {
  const accounts = await ethers.getSigners();

  for (const account of accounts) {
    console.log(account.address);
  }
});

// You need to export an object to set up your config
// Go to https://hardhat.org/config/ to learn more

```

```
/**
 * @type import('hardhat/config').HardhatUserConfig
 */
module.exports = {
  solidity: "0.7.3",
  networks: {
    sepolia: {
      chainId: 11155111
    }
  },
  privateKeys: ['YOUR_PRIVATE_KEY_HERE'] // Replace with your actual private key
};

```

- Vào Metamask chọn mũi trên hướng xuống cạnh account và chọn dấu 3 chấm để “Account details” => “Private key” => Nhập password => enter => Copy private key=> gán vào ‘YOUR_PRIVATE_KEY_HERE’ trong file config.
- Sửa file sample-script.js

```
// We require the Hardhat Runtime Environment explicitly here. This is optional
// but useful for running the script in a standalone fashion through `node <script>`.
//
// When running the script with `hardhat run <script>` you'll find the Hardhat
// Runtime Environment's members available in the global scope.
const hre = require("hardhat");

async function main() {
  // Hardhat always runs the compile task when running scripts with its command
  // line interface.
  //
  // If this script is run directly using `node` you may want to call compile
  // manually to make sure everything is compiled
```

```
  // await hre.run('compile');

  // We get the contract to deploy

  const FirstSmartContract = await
hre.ethers.getContractFactory("FirstSmartContract");
  const firstSmartContract = await FirstSmartContract.deploy();

  await firstSmartContract.deployed();

  console.log("FirstSmartContract deployed to:",
firstSmartContract.address);
}

// We recommend this pattern to be able to use async/await
everywhere
// and properly handle errors.
main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error(error);
    process.exit(1);
  });
```

- Chạy câu lệnh deploy: npx hardhat run scripts/sample-script.js --network sepolia

```
D:\IT\SmartContract\smartcontract>npx hardhat run scripts/sample-script.js --network sepolia
FirstSmartContract deployed to: 0xd188E6D891424Aabda65B506e0De2B18D9c6B6d5
```

Bài thực hành:

- Deploy Smartcontract DominionDAO

- Code: https://github.com/VuManh-07/CodeSmartcontractThucHanhPtit

Bài 2 Tạo Web3

1. Khởi tạo dự án với React

Bước 1. Cài đặt Node.js và npm

- Trước tiên, bạn cần cài đặt Node.js và npm trên máy tính của mình nếu chưa có. Node.js là một nền tảng phát triển dựa trên JavaScript, cho phép bạn chạy mã JavaScript trên máy tính của mình. npm là một trình quản lý gói cho Node.js, cho phép bạn cài đặt các gói JavaScript từ trung tâm lưu trữ npm.
- Bạn có thể tải Node.js và npm từ trang chủ của Node.js tại địa chỉ: https://nodejs.org

Bước 2. Cài đặt Create React App

- npm install -g create-react-app

- Lệnh trên sẽ cài đặt Create React App trên máy tính của bạn.

Bước 3. Tạo dự án React

- Sau khi cài đặt Create React App, bạn có thể tạo dự án React bằng cách mở Terminal hoặc Command Prompt và chạy lệnh sau:

- npx create-react-app web3-app

- Lệnh trên sẽ tạo ra một dự án React mới có tên là "web3-app". Bạn có thể thay đổi tên của dự án bằng cách thay đổi "web3-app" thành tên bạn muốn.

- Sau khi chạy lệnh trên, Create React App sẽ tạo ra một dự án React mới với cấu trúc thư mục và tệp tin chuẩn. Bạn có thể mở dự án bằng cách sử dụng Terminal hoặc Command Prompt và chạy các lệnh sau:

- cd my-app
- npm start

- Lệnh trên sẽ mở trình duyệt và chạy ứng dụng React của bạn trên địa chỉ http://localhost:3000

2. Cài đặt và cấu hình các packages

- Vào trong file package.json của dự án và thay bằng code dưới đây, sau đó chạy câu lệnh: **npm install**

- Code package.json:

```
{
  "name": "web3-app",
  "private": true,
  "version": "0.1.0",
  "scripts": {
    "start": "react-app-rewired start",
    "build": "react-app-rewired build",
    "test": "react-app-rewired test",
    "eject": "react-scripts eject"
  },
  "dependencies": {
    "@cometchat-pro/chat": "3.0.6",
    "moment": "^2.29.3",
    "react": "^17.0.2",
    "react-dom": "^17.0.2",
    "react-hooks-global-state": "^1.0.2",
    "react-icons": "^4.3.1",
    "react-identicons": "^1.2.5",
    "react-moment": "^1.1.2",
    "react-router-dom": "6",
    "react-scripts": "5.0.0",
    "react-toastify": "^9.0.1",
    "recharts": "^2.1.9",
    "web-vitals": "^2.1.4",
    "web3": "^1.7.1"
  },
  "devDependencies": {
    "@openzeppelin/contracts": "^4.5.0",
    "@tailwindcss/forms": "0.4.0",
    "@truffle/hdwallet-provider": "^2.0.4",
    "assert": "^2.0.0",
    "autoprefixer": "10.4.2",
    "babel-polyfill": "^6.26.0",
    "babel-preset-env": "^1.7.0",
    "babel-preset-es2015": "^6.24.1",
    "babel-preset-stage-2": "^6.24.1",
    "babel-preset-stage-3": "^6.24.1",
    "babel-register": "^6.26.0",
    "buffer": "^6.0.3",
    "chai": "^4.3.6",
    "chai-as-promised": "^7.1.1",
```

```
    "crypto-browserify": "^3.12.0",
    "dotenv": "^16.0.0",
    "https-browserify": "^1.0.0",
    "mnemonics": "^1.1.3",
    "os-browserify": "^0.3.0",
    "postcss": "8.4.5",
    "process": "^0.11.10",
    "react-app-rewired": "^2.1.11",
    "stream-browserify": "^3.0.0",
    "stream-http": "^3.2.0",
    "tailwindcss": "3.0.18",
    "url": "^0.11.0"
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all"
    ],
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
  }
}
```

- Tiếp theo thêm các file config bằng cách tạo chọn new file:

+ config-overrides.js

```
// This file is included to deal with issues regarding react-scripts
v5.0.0 and web3
// Provided from Web3 documentation ->
https://github.com/ChainSafe/web3.js#troubleshooting-and-known-issues

const webpack = require('webpack');

module.exports = function override(config) {
    const fallback = config.resolve.fallback || {};
    Object.assign(fallback, {
```

```
        "crypto": require.resolve("crypto-browserify"),
        "stream": require.resolve("stream-browserify"),
        "assert": require.resolve("assert"),
        "http": require.resolve("stream-http"),
        "https": require.resolve("https-browserify"),
        "os": require.resolve("os-browserify"),
        "url": require.resolve("url")
    })
    config.resolve.fallback = fallback;
    config.plugins = (config.plugins || []).concat([
        new webpack.ProvidePlugin({
            process: 'process/browser',
            Buffer: ['buffer', 'Buffer']
        })
    ])
    return config;
}
```

+ tailwind.config.js

```
module.exports = {
  content: ['./src/**/*.{js,jsx,ts,tsx}'],
  mode: 'jit',
  darkMode: 'class', // or 'media' or 'class'
  theme: {
    fontFamily: {
      display: ['Open Sans', 'sans-serif'],
      body: ['Open Sans', 'sans-serif'],
    },
    extend: {
      screens: {
        mf: '990px',
      },
      keyframes: {
        'slide-in': {
          '0%': {
            '-webkit-transform': 'translateX(120%)',
            transform: 'translateX(120%)',
          },
          '100%': {
```

```
            '-webkit-transform': 'translateX(0%)',
            transform: 'translateX(0%)',
          },
        },
      },
      animation: {
        'slide-in': 'slide-in 0.5s ease-out',
      },
    },
  },
  variants: {
    extend: {},
  },
  plugins: [require('@tailwindcss/forms')],
}
```

+ postcss.config.js

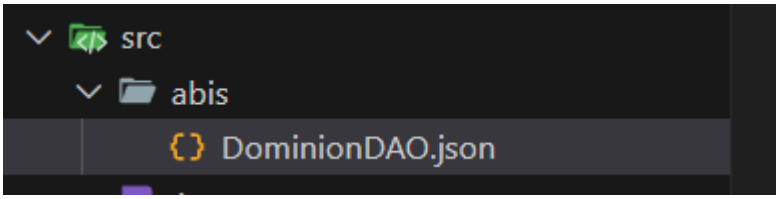
```
module.exports = {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

+ .env (file biến môi trường): ** là địa chỉ address contract đã deploy ở phần TH trên

```
REACT_APP_CONTRACT_ADDRESS=**
```

3. Tạo dự án

- Tạo folder “abis” trong “src” và copy “DominionDAO.json” trong artifacts\contracts\DominionDAO.sol vừa deploy Smartcontract ở phía trên.



- Tạo folder “store” trong “src” sau đó tạo file index.jsx và thêm code sau:

```
import { createGlobalState } from 'react-hooks-global-state'
import moment from 'moment'

const { setGlobalState, useGlobalState, getGlobalState } =
createGlobalState({
  createModal: 'scale-0',
  connectedAccount: '',
  contract: null,
  proposals: [],
  isStakeholder: false,
  balance: 0,
  mybalance: 0,
})

const truncate = (text, startChars, endChars, maxLength) => {
  if (text.length > maxLength) {
    var start = text.substring(0, startChars)
    var end = text.substring(text.length - endChars, text.length)
    while (start.length + end.length < maxLength) {
      start = start + '.'
    }
    return start + end
  }
  return text
}

const daysRemaining = (days) => {
  const todaysdate = moment()
  days = Number((days + '000').slice(0))
  days = moment(days).format('YYYY-MM-DD')
  days = moment(days)
  days = days.diff(todaysdate, 'days')
  return days == 1 ? '1 day' : days + ' days'
}

export {
  truncate,
  setGlobalState,
  useGlobalState,
  getGlobalState,
  daysRemaining,
```

```
}
```

- Tạo file “Blockchain.services.jsx” trong “src”:

```
import Web3 from "web3";
import { setGlobalState, getGlobalState } from "./store";
import abi from "./abis/DominionDAO.json";

const { ethereum } = window;
window.web3 = new Web3(ethereum);
window.web3 = new Web3(window.web3.currentProvider);

const connectWallet = async () => {
  try {
    if (!ethereum) return alert("Please install Metamask");
    const accounts = await ethereum.request({ method:
"eth_requestAccounts" });
    setGlobalState("connectedAccount", accounts[0].toLowerCase());
  } catch (error) {
    reportError(error);
  } finally {
    window.location.reload();
  }
};

const isWallectConnected = async () => {
  try {
    if (!ethereum) return alert("Please install Metamask");
    const accounts = await ethereum.request({ method: "eth_accounts" });

    window.ethereum.on("chainChanged", (chainId) => {
      window.location.reload();
    });

    window.ethereum.on("accountsChanged", async () => {
      if (accounts.length) {
        setGlobalState("connectedAccount", accounts[0].toLowerCase());
        await isWallectConnected();
      }
    });
```

```
    if (accounts.length) {
      setGlobalState("connectedAccount", accounts[0].toLowerCase());
    } else {
      alert("Please connect wallet.");
      console.log("No accounts found.");
    }
  } catch (error) {
    reportError(error);
  }
};

const getEtheriumContract = async () => {
  const connectedAccount = getGlobalState("connectedAccount");

  if (connectedAccount) {
    const web3 = window.web3;
    const contractAddress = process.env.REACT_APP_CONTRACT_ADDRESS;
    console.log("address:", contractAddress);
    const contract = new web3.eth.Contract(abi.abi, contractAddress);
    return contract;
  } else {
    return getGlobalState("contract");
  }
};

const performContribute = async (amount) => {
  try {
    amount = window.web3.utils.toWei(amount.toString(), "ether");
    const contract = await getEtheriumContract();
    const account = getGlobalState("connectedAccount");

    await contract.methods.contribute().send({ from: account, value:
amount });

    window.location.reload();
  } catch (error) {
    reportError(error);
    return error;
  }
};
```

```
const getInfo = async () => {
  try {
    if (!ethereum) return alert("Please install Metamask");

    const contract = await getEtheriumContract();
    const connectedAccount = getGlobalState("connectedAccount");
    const isStakeholder = await contract.methods
      .isStakeholder()
      .call({ from: connectedAccount });
    const balance = await contract.methods.daoBalance().call();
    const mybalance = await contract.methods
      .getBalance()
      .call({ from: connectedAccount });
    setGlobalState("balance", window.web3.utils.fromWei(balance));
    setGlobalState("mybalance", window.web3.utils.fromWei(mybalance));
    setGlobalState("isStakeholder", isStakeholder);
  } catch (error) {
    reportError(error);
  }
};

const raiseProposal = async ({ title, description, beneficiary, amount })
=> {
  try {
    amount = window.web3.utils.toWei(amount.toString(), "ether");
    const contract = await getEtheriumContract();
    const account = getGlobalState("connectedAccount");

    await contract.methods
      .createProposal(title, description, beneficiary, amount)
      .send({ from: account });

    window.location.reload();
  } catch (error) {
    reportError(error);
    return error;
  }
};

const getProposals = async () => {
  try {
```

```
    if (!ethereum) return alert("Please install Metamask");

    const contract = await getEtheriumContract();
    const proposals = await contract.methods.getProposals().call();
    setGlobalState("proposals", structuredProposals(proposals));
  } catch (error) {
    reportError(error);
  }
};

const structuredProposals = (proposals) => {
  return proposals
    .map((proposal) => ({
      id: proposal.id,
      amount: window.web3.utils.fromWei(proposal.amount),
      title: proposal.title,
      description: proposal.description,
      paid: proposal.paid,
      passed: proposal.passed,
      proposer: proposal.proposer,
      upvotes: Number(proposal.upvotes),
      downvotes: Number(proposal.downvotes),
      beneficiary: proposal.beneficiary,
      executor: proposal.executor,
      duration: proposal.duration,
    }))
    .reverse();
};

const getProposal = async (id) => {
  try {
    const proposals = getGlobalState("proposals");
    return proposals.find((proposal) => proposal.id == id);
  } catch (error) {
    reportError(error);
  }
};

const voteOnProposal = async (proposalId, supported) => {
  try {
    const contract = await getEtheriumContract();
```

```
    const account = getGlobalState("connectedAccount");
    await contract.methods
      .performVote(proposalId, supported)
      .send({ from: account });

    window.location.reload();
  } catch (error) {
    reportError(error);
  }
};

const listVoters = async (id) => {
  try {
    const contract = await getEtheriumContract();
    const votes = await contract.methods.getVotesOf(id).call();
    return votes;
  } catch (error) {
    reportError(error);
  }
};

const payoutBeneficiary = async (id) => {
  try {
    const contract = await getEtheriumContract();
    const account = getGlobalState("connectedAccount");
    await contract.methods.payBeneficiary(id).send({ from: account });
    window.location.reload();
  } catch (error) {
    reportError(error);
  }
};

const reportError = (error) => {
  console.log(JSON.stringify(error), "red");
  // throw new Error("No ethereum object.");
};

export {
  isWallectConnected,
  connectWallet,
  performContribute,
```

```
  getInfo,
  raiseProposal,
  getProposals,
  getProposal,
  voteOnProposal,
  listVoters,
  payoutBeneficiary,
};
```

- Tạo folder “components” trong “src” với các file .jsx sau:

+ Banner.jsx

```
import { useState } from 'react'
import { useGlobalState, setGlobalState } from '../store'
import { toast } from 'react-toastify'
import { performContribute } from '../Blockchain.services'

const Banner = () => {
  const [isStakeholder] = useGlobalState('isStakeholder')
  const [proposals] = useGlobalState('proposals')
  const [balance] = useGlobalState('balance')
  const [mybalance] = useGlobalState('mybalance')
  const [amount, setAmount] = useState('')

  const onPropose = () => {
    if (!isStakeholder) return
    setGlobalState('createModal', 'scale-100')
  }

  const onContribute = async () => {
    if (!!!amount || amount == '') return
    await performContribute(amount)
    toast.success('Contribution received')
  }

  const opened = () =>
    proposals.filter(
      (proposal) => new Date().getTime() < Number(proposal.duration +
'000'),
    ).length
```

```
  return (
    <div className="p-8">
      <h2 className="font-semibold text-3xl mb-5">
        {opened()} Proposal{opened() == 1 ? '' : 's'} Currently Opened
      </h2>
      <p>
        Current DAO Balance: <strong>{balance} Eth</strong> <br />
        Your contributions:{' '}
        <span>
          <strong>{mybalance} Eth</strong>
          {isStakeholder ? ', and you are now a stakeholder 😊' : null}
        </span>
      </p>
      <hr className="my-6 border-gray-300 dark:border-gray-500" />
      <p>
        {isStakeholder
          ? 'You can now raise proposals on this platform 😆'
          : 'Hey, when you contribute upto 1 ether you become a
stakeholder 😇'}
      </p>
      <div className="flex flex-row justify-start items-center md:w-1/3
w-full mt-4">
        <input
          type="number"
          className="form-control block w-full px-3 py-1.5
          text-base font-normaltext-gray-700
          bg-clip-padding border border-solid border-gray-300
          rounded transition ease-in-out m-0 shadow-md
          focus:text-gray-500 focus:outline-none
          dark:border-gray-500 dark:bg-transparent"
          placeholder="e.g 2.5 Eth"
          onChange={(e) => setAmount(e.target.value)}
          value={amount}
          required
        />
      </div>
      <div
        className="flex flex-row justify-start items-center space-x-3 mt-
4"
        role="group"
```

```
      >
        <button
          type="button"
          className={`inline-block px-6 py-2.5
          bg-blue-600 text-white font-medium text-xs
          leading-tight uppercase shadow-md rounded-full
          hover:bg-blue-700 hover:shadow-lg focus:bg-blue-700
          focus:shadow-lg focus:outline-none focus:ring-0
          active:bg-blue-800 active:shadow-lg transition
          duration-150 ease-in-out dark:text-blue-500
          dark:border dark:border-blue-500 dark:bg-transparent`}
          data-mdb-ripple="true"
          data-mdb-ripple-color="light"
          onClick={onContribute}
        >
          Contribute
        </button>

        {isStakeholder ? (
          <button
            type="button"
            className={`inline-block px-6 py-2.5
            bg-blue-600 text-white font-medium text-xs
            leading-tight uppercase shadow-md rounded-full
            hover:bg-blue-700 hover:shadow-lg focus:bg-blue-700
            focus:shadow-lg focus:outline-none focus:ring-0
            active:bg-blue-800 active:shadow-lg transition
            duration-150 ease-in-out dark:text-blue-500
            dark:border dark:border-blue-500 dark:bg-transparent`}
            data-mdb-ripple="true"
            data-mdb-ripple-color="light"
            onClick={onPropose}
          >
            Propose
          </button>
        ) : null}
      </div>
    </div>
  )
}
```

```
export default Banner
```

+ CreateProposal.jsx

```
import { useState } from 'react'
import { FaTimes } from 'react-icons/fa'
import { setGlobalState, useGlobalState } from '../store'
import { raiseProposal } from '../Blockchain.services'
import { toast } from 'react-toastify'

const CreateProposal = () => {
  const [createModal] = useGlobalState('createModal')
  const [title, setTitle] = useState('')
  const [amount, setAmount] = useState('')
  const [beneficiary, setBeneficiary] = useState('')
  const [description, setDescription] = useState('')

  const handleSubmit = async (e) => {
    e.preventDefault()
    if (!title || !description || !beneficiary || !amount) return
    const proposal = { title, description, beneficiary, amount }

    await raiseProposal(proposal)
    toast.success('Proposal created, reloading in progress...')
    closeModal()
  }

  const closeModal = () => {
    setGlobalState('createModal', 'scale-0')
    resetForm()
  }

  const resetForm = () => {
    setTitle('')
    setAmount('')
    setBeneficiary('')
    setDescription('')
  }
```

```
  return (
    <div
      className={`fixed top-0 left-0 w-screen h-screen flex items-center
      justify-center bg-black bg-opacity-50 transform z-50
      transition-transform duration-300 ${createModal}`}
    >
      <div className="bg-white dark:bg-[#212936] shadow-xl shadow-
[#122643] dark:shadow-gray-500 rounded-xl w-11/12 md:w-2/5 h-7/12 p-6">
        <form className="flex flex-col">
          <div className="flex flex-row justify-between items-center">
            <p className="font-semibold">Raise Proposal</p>
            <button
              type="button"
              onClick={closeModal}
              className="border-0 bg-transparent focus:outline-none"
            >
              <FaTimes />
            </button>
          </div>

          <div className="flex flex-row justify-between items-center
border border-gray-500 dark:border-gray-500 rounded-xl mt-5">
            <input
              className="block w-full text-sm
              bg-transparent border-0
              focus:outline-none focus:ring-0"
              type="text"
              name="title"
              placeholder="Title"
              onChange={(e) => setTitle(e.target.value)}
              value={title}
              required
            />
          </div>

          <div className="flex flex-row justify-between items-center
border border-gray-500 dark:border-gray-500 rounded-xl mt-5">
            <input
              className="block w-full text-sm
              bg-transparent border-0
              focus:outline-none focus:ring-0"
```

```
            type="text"
            name="amount"
            placeholder="e.g 2.5 Eth"
            onChange={(e) => setAmount(e.target.value)}
            value={amount}
            required
          />
        </div>

        <div className="flex flex-row justify-between items-center
border border-gray-500 dark:border-gray-500 rounded-xl mt-5">
          <input
            className="block w-full text-sm
            bg-transparent border-0
            focus:outline-none focus:ring-0"
            type="text"
            name="beneficiary"
            placeholder="Beneficiary Address"
            onChange={(e) => setBeneficiary(e.target.value)}
            value={beneficiary}
            required
          />
        </div>

        <div className="flex flex-row justify-between items-center
border border-gray-500 dark:border-gray-500 rounded-xl mt-5">
          <textarea
            className="block w-full text-sm resize-none
            bg-transparent border-0
            focus:outline-none focus:ring-0 h-20"
            type="text"
            name="description"
            placeholder="Description"
            onChange={(e) => setDescription(e.target.value)}
            value={description}
            required
          ></textarea>
        </div>

        <button
          className="rounded-lg px-6 py-2.5 bg-blue-600
```

```
              text-white font-medium text-xs leading-tight
              uppercase hover:bg-blue-700 focus:bg-blue-700
              focus:outline-none focus:ring-0 active:bg-blue-800
              transition duration-150 ease-in-out mt-5"
            onClick={handleSubmit}
          >
            Submit Proposal
          </button>
        </form>
      </div>
    </div>
  )
}

export default CreateProposal
```

+ Header.jsx

```
import { Link } from "react-router-dom";
import { MdLightMode } from "react-icons/md";
import { FaUserSecret, FaMoon } from "react-icons/fa";
import { useEffect, useState } from "react";
import { truncate, useGlobalState } from "../store";
import { connectWallet } from "../Blockchain.services";

const Header = () => {
  const [theme, setTheme] = useState(localStorage.theme);
  const themeColor = theme === "dark" ? "light" : "dark";
  const darken = theme === "dark" ? true : false;
  const [connectedAccount] = useGlobalState("connectedAccount");

  useEffect(() => {
    const root = window.document.documentElement;
    root.classList.remove(themeColor);
    root.classList.add(theme);
    localStorage.setItem("theme", theme);
  }, [themeColor, theme]);

  const toggleLight = () => {
```

```
    const root = window.document.documentElement;
    root.classList.remove(themeColor);
    root.classList.add(theme);
    localStorage.setItem("theme", theme);
    setTheme(themeColor);
  };

  return (
    <header className="sticky top-0 z-50 dark:text-blue-500">
      <nav
        className="navbar navbar-expand-lg shadow-md py-2 relative flex
items-center w-full justify-between
      bg-white dark:bg-[#212936]"
      >
        <div className="px-6 w-full flex flex-wrap items-center justify-
between">
          <div className="grow flex flex-row justify-between items-center
p-2">
            <Link
              to={"/"}
              className="flex flex-row justify-start items-center space-
x-3"
            >
              <img
                src="https://haitrieu.com/wp-
content/uploads/2021/10/Logo-Hoc-Vien-Cong-Nghe-Buu-Chinh-Vien-Thong-
PTITSimple-635x807.png"
                alt="logo"
              />
              <span className="invisible md:visible dark:text-gray-300
font-bold text-xl">
                PTIT Dominion
              </span>
            </Link>

            <div className="flex flex-row justify-center items-center
space-x-5">
              {darken ? (
                <MdLightMode
                  className="cursor-pointer"
```

```
                size={25}
                onClick={toggleLight}
              />
            ) : (
              <FaMoon
                className="cursor-pointer"
                size={25}
                onClick={toggleLight}
              />
            )}

            {connectedAccount ? (
              <button
                className="px-4 py-2.5 bg-blue-600 text-white
                font-medium text-xs leading-tight uppercase
                rounded-full shadow-md hover:bg-blue-700 hover:shadow-
lg

                focus:bg-blue-700 focus:shadow-lg focus:outline-none
                focus:ring-0 active:bg-blue-800 active:shadow-lg
                transition duration-150 ease-in-out dark:text-blue-500
                dark:border dark:border-blue-500 dark:bg-transparent"
              >
                {truncate(connectedAccount, 4, 4, 11)}
              </button>
            ) : (
              <button
                className="px-4 py-2.5 bg-blue-600 text-white
                font-medium text-xs leading-tight uppercase
                rounded-full shadow-md hover:bg-blue-700 hover:shadow-
lg

                focus:bg-blue-700 focus:shadow-lg focus:outline-none
                focus:ring-0 active:bg-blue-800 active:shadow-lg
                transition duration-150 ease-in-out dark:text-blue-500
                dark:border dark:border-blue-500 dark:bg-transparent"
                onClick={connectWallet}
              >
                Connect Wallet
              </button>
            )}
          </div>
        </div>
```

```
        </div>
      </nav>
    </header>
  );
};

export default Header;
```

+ ProposalDetails.jsx

```
import { useEffect, useState } from 'react'
import { useParams } from 'react-router-dom'
import { toast } from 'react-toastify'
import {
  BarChart,
  Bar,
  CartesianGrid,
  XAxis,
  YAxis,
  Legend,
  Tooltip,
} from 'recharts'
import { getProposal, voteOnProposal } from '../Blockchain.services'
import { useGlobalState, daysRemaining } from '../store'

const ProposalDetails = () => {
  const { id } = useParams()
  const [proposal, setProposal] = useState(null)
  const [data, setData] = useState([])
  const [isStakeholder] = useGlobalState('isStakeholder')

  useEffect(() => {
    retrieveProposal()
  }, [id])

  const retrieveProposal = async () => {
    await getProposal(id).then((res) => {
      setProposal(res)
      setData([
        {
```

```
          name: 'Voters',
          Acceptees: res?.upvotes,
          Rejectees: res?.downvotes,
        },
      ])
    })
  }

  const onVote = async (choice) => {
    if (new Date().getTime() > Number(proposal.duration + '000')) {
      toast.warning('Proposal expired!')
      return
    }

    await voteOnProposal(id, choice)
    toast.success('Voted successfully!')
  }

  return (
    <div className="p-8">
      <h2 className="font-semibold text-3xl mb-5">{proposal?.title}</h2>
      <p>
        This proposal is to payout <strong>{proposal?.amount}
Eth</strong> and
        currently have{' '}
        <strong>{proposal?.upvotes + proposal?.downvotes} votes</strong>
and
        will expire in
<strong>{daysRemaining(proposal?.duration)}</strong>
      </p>
      <hr className="my-6 border-gray-300" />
      <p>{proposal?.description}</p>
      <div className="flex flex-row justify-start items-center w-full mt-
4 overflow-auto">
        <BarChart width={730} height={250} data={data}>
          <CartesianGrid strokeDasharray="3 3" />
          <XAxis dataKey="name" />
          <YAxis />
          <Tooltip />
          <Legend />
          <Bar dataKey="Acceptees" fill="#2563eb" />
```

```
          <Bar dataKey="Rejectees" fill="#dc2626" />
        </BarChart>
      </div>
      <div
        className="flex flex-row justify-start items-center space-x-3 mt-
4"
        role="group"
      >
        {isStakeholder ? (
          <>
            <button
              type="button"
              className="inline-block px-6 py-2.5
            bg-blue-600 text-white font-medium text-xs
              leading-tight uppercase rounded-full shadow-md
              hover:bg-blue-700 hover:shadow-lg focus:bg-blue-700
              focus:shadow-lg focus:outline-none focus:ring-0
              active:bg-blue-800 active:shadow-lg transition
              duration-150 ease-in-out dark:text-gray-300
              dark:border dark:border-gray-500 dark:bg-transparent"
              data-mdb-ripple="true"
              data-mdb-ripple-color="light"
              onClick={() => onVote(true)}
            >
              Accept
            </button>
            <button
              type="button"
              className="inline-block px-6 py-2.5
            bg-blue-600 text-white font-medium text-xs
              leading-tight uppercase rounded-full shadow-md
              hover:bg-blue-700 hover:shadow-lg focus:bg-blue-700
              focus:shadow-lg focus:outline-none focus:ring-0
              active:bg-blue-800 active:shadow-lg transition
              duration-150 ease-in-out
              dark:border dark:border-gray-500 dark:bg-transparent"
              data-mdb-ripple="true"
              data-mdb-ripple-color="light"
              onClick={() => onVote(false)}
            >
              Reject
```

```
            </button>
          </>
        ) : null}
      </div>
    </div>
  )
}

export default ProposalDetails
```

+ Proposals.jsx

```
import Identicon from 'react-identicons'
import { useState } from 'react'
import { Link } from 'react-router-dom'
import { truncate, useGlobalState, daysRemaining } from '../store'
import { payoutBeneficiary } from "../Blockchain.services";
import { toast } from 'react-toastify'

const Proposals = () => {
  const [data] = useGlobalState('proposals')
  const [proposals, setProposals] = useState(data)

  const deactive = `bg-transparent
  text-blue-600 font-medium text-xs leading-tight
  uppercase hover:bg-blue-700 focus:bg-blue-700
  focus:outline-none focus:ring-0 active:bg-blue-600
  transition duration-150 ease-in-out overflow-hidden
  border border-blue-600 hover:text-white focus:text-white`

  const active = `bg-blue-600
  text-white font-medium text-xs leading-tight
  uppercase hover:bg-blue-700 focus:bg-blue-700
  focus:outline-none focus:ring-0 active:bg-blue-800
  transition duration-150 ease-in-out overflow-hidden
  border border-blue-600`

  const getAll = () => setProposals(data)
```

```
  const getOpened = () =>
    setProposals(
      data.filter(
        (proposal) => new Date().getTime() < Number(proposal.duration +
'000')
      )
    )

  const getClosed = () =>
    setProposals(
      data.filter(
        (proposal) => new Date().getTime() > Number(proposal.duration +
'000')
      )
    )

  const handlePayout = async (id) => {
    await payoutBeneficiary(id)
    toast.success("Beneficiary successfully Paid Out!");
  }

  return (
    <div className="flex flex-col p-8">
      <div className="flex flex-row justify-center items-center"
role="group">
        <button
          aria-current="page"
          className={`rounded-l-full px-6 py-2.5 ${active}`}
          onClick={getAll}
        >
          All
        </button>
        <button
          aria-current="page"
          className={`px-6 py-2.5 ${deactive}`}
          onClick={getOpened}
        >
          Open
        </button>
        <button
          aria-current="page"
```

```
          className={`rounded-r-full px-6 py-2.5 ${deactive}`}
          onClick={getClosed}
        >
          Closed
        </button>
      </div>
      <div className="overflow-x-auto sm:-mx-6 lg:-mx-8">
        <div className="py-2 inline-block min-w-full sm:px-6 lg:px-8">
          <div className="h-[calc(100vh_-_20rem)] overflow-y-auto shadow-
md rounded-md">
            <table className="min-w-full">
              <thead className="border-b dark:border-gray-500">
                <tr>
                  <th
                    scope="col"
                    className="text-sm font-medium px-6 py-4 text-left"
                  >
                    Created By
                  </th>
                  <th
                    scope="col"
                    className="text-sm font-medium px-6 py-4 text-left"
                  >
                    Title
                  </th>
                  <th
                    scope="col"
                    className="text-sm font-medium px-6 py-4 text-left"
                  >
                    Expires
                  </th>
                  <th
                    scope="col"
                    className="text-sm font-medium px-6 py-4 text-left"
                  >
                    Action
                  </th>
                </tr>
              </thead>
              <tbody>
                {proposals.map((proposal) => (
```

```
                <tr
                  key={proposal.id}
                  className="border-b dark:border-gray-500"
                >
                  <td className="text-sm font-light px-6 py-4
whitespace-nowrap">
                    <div className="flex flex-row justify-start items-
center space-x-3">
                      <Identicon
                        string={proposal.proposer.toLowerCase()}
                        size={25}
                        className="h-10 w-10 object-contain rounded-
full mr-3"
                      />
                      <span>{truncate(proposal.proposer, 4, 4,
11)}</span>
                    </div>
                  </td>
                  <td className="text-sm font-light px-6 py-4
whitespace-nowrap">
                    {proposal.title.substring(0, 80) + '...'}
                  </td>
                  <td className="text-sm font-light px-6 py-4
whitespace-nowrap">
                    {new Date().getTime() > Number(proposal.duration +
'000')
                      ? 'Expired'
                      : daysRemaining(proposal.duration)}
                  </td>
                  <td
                    className="flex justify-start items-center space-x-
3
                    text-sm font-light px-6 py-4 whitespace-nowrap"
                  >
                    <Link
                      to={'/proposal/' + proposal.id}
                      className="dark:border rounded-full px-6 py-2.5
dark:border-blue-600
                        dark:text-blue-600 dark:bg-transparent font-
medium text-xs leading-tight
```

```
                        uppercase hover:border-blue-700 focus:border-
blue-700
                        focus:outline-none focus:ring-0 active:border-
blue-800
                        transition duration-150 ease-in-out text-white
bg-blue-600"
                    >
                      View
                    </Link>

                    {new Date().getTime() >
                    Number(proposal.duration + '000') ? (
                      proposal.upvotes > proposal.downvotes ? (
                        !proposal.paid ? (
                          <button
                            className="dark:border rounded-full px-6
py-2.5 dark:border-red-600
                              dark:text-red-600 dark:bg-transparent
font-medium text-xs leading-tight
                              uppercase hover:border-red-700
focus:border-red-700
                              focus:outline-none focus:ring-0
active:border-red-800
                              transition duration-150 ease-in-out text-
white bg-red-600"
                            onClick={() => handlePayout(proposal.id)}
                          >
                            Payout
                          </button>
                        ) : (
                          <button
                            className="dark:border rounded-full px-6
py-2.5 dark:border-green-600
                              dark:text-green-600 dark:bg-transparent
font-medium text-xs leading-tight
                              uppercase hover:border-green-700
focus:border-green-700
                              focus:outline-none focus:ring-0
active:border-green-800
                              transition duration-150 ease-in-out
text-white bg-green-600"
```

```
                            >
                              Paid
                            </button>
                          )
                        ) : (
                          <button
                              className="dark:border rounded-full px-6
py-2.5 dark:border-red-600
                                  dark:text-red-600 dark:bg-transparent
font-medium text-xs leading-tight
                                  uppercase hover:border-red-700
focus:border-red-700
                                  focus:outline-none focus:ring-0
active:border-red-800
                                  transition duration-150 ease-in-out
text-white bg-red-600"
                            >
                              Rejected
                            </button>
                        )
                      ) : null}
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </div>
      </div>
    </div>
  )
}

export default Proposals
```

+ Voters.jsx

```
import Identicon from 'react-identicons'
import moment from 'moment'
```

```
import { useState, useEffect } from 'react'
import { useParams } from 'react-router-dom'
import { truncate } from '../store'
import { listVoters } from '../Blockchain.services'

const Voters = () => {
  const [voters, setVoters] = useState([])
  const [data, setData] = useState([])
  const { id } = useParams()

  const timeAgo = (timestamp) => moment(Number(timestamp +
'000')).fromNow()

  const deactive = `bg-transparent
  text-blue-600 font-medium text-xs leading-tight
  uppercase hover:bg-blue-700 focus:bg-blue-700
  focus:outline-none focus:ring-0 active:bg-blue-600
  transition duration-150 ease-in-out overflow-hidden
  border border-blue-600 hover:text-white focus:text-white`

  const active = `bg-blue-600
  text-white font-medium text-xs leading-tight
  uppercase hover:bg-blue-700 focus:bg-blue-700
  focus:outline-none focus:ring-0 active:bg-blue-800
  transition duration-150 ease-in-out overflow-hidden
  border border-blue-600`

  useEffect(async () => {
    await listVoters(id).then((res) => {
      setVoters(res)
      setData(res)
    })
  }, [id])

  const getAll = () => setVoters(data)

  const getAccepted = () => setVoters(data.filter((vote) =>
vote.choosen))

  const getRejected = () => setVoters(data.filter((vote) =>
!vote.choosen))
```

```
  return (
    <div className="flex flex-col p-8">
      <div className="flex flex-row justify-center items-center"
role="group">
        <button
          aria-current="page"
          className={`rounded-l-full px-6 py-2.5 ${active}`}
          onClick={getAll}
        >
          All
        </button>
        <button
          aria-current="page"
          className={`px-6 py-2.5 ${deactive}`}
          onClick={getAccepted}
        >
          Acceptees
        </button>
        <button
          aria-current="page"
          className={`rounded-r-full px-6 py-2.5 ${deactive}`}
          onClick={getRejected}
        >
          Rejectees
        </button>
      </div>
      <div className="overflow-x-auto sm:-mx-6 lg:-mx-8">
        <div className="py-2 inline-block min-w-full sm:px-6 lg:px-8">
          <div className="h-[calc(100vh_-_20rem)] overflow-y-
auto  shadow-md rounded-md">
            <table className="min-w-full">
              <thead className="border-b dark:border-gray-500">
                <tr>
                  <th
                    scope="col"
                    className="text-sm font-medium px-6 py-4 text-left"
                  >
                    Voter
                  </th>
                  <th
```

```
                  scope="col"
                  className="text-sm font-medium px-6 py-4 text-left"
                >
                  Voted
                </th>
                <th
                  scope="col"
                  className="text-sm font-medium px-6 py-4 text-left"
                >
                  Vote
                </th>
              </tr>
            </thead>
            <tbody>
              {voters.map((voter, i) => (
                <tr
                  key={i}
                  className="border-b dark:border-gray-500 transition
duration-300 ease-in-out"
                >
                  <td className="text-sm font-light px-6 py-4
whitespace-nowrap">
                    <div className="flex flex-row justify-start items-
center space-x-3">
                      <Identicon
                        string={voter.voter.toLowerCase()}
                        size={25}
                        className="h-10 w-10 object-contain rounded-
full mr-3"
                      />
                      <span>{truncate(voter.voter, 4, 4, 11)}</span>
                    </div>
                  </td>
                  <td className="text-sm font-light px-6 py-4
whitespace-nowrap">
                    {timeAgo(voter.timestamp)}
                  </td>
                  <td className="text-sm font-light px-6 py-4
whitespace-nowrap">
                    {voter.choosen ? (
                      <button
```

```
                        className="border-2 rounded-full px-6 py-2.5
border-blue-600
                        text-blue-600 font-medium text-xs leading-tight
                        uppercase hover:border-blue-700 focus:border-
blue-700
                        focus:outline-none focus:ring-0 active:border-
blue-800
                        transition duration-150 ease-in-out"
                      >
                        Accepted
                      </button>
                    ) : (
                      <button
                        className="border-2 rounded-full px-6 py-2.5
border-red-600
                        text-red-600 font-medium text-xs leading-tight
                        uppercase hover:border-red-700 focus:border-
red-700
                        focus:outline-none focus:ring-0 active:border-
red-800
                        transition duration-150 ease-in-out"
                      >
                        Rejected
                      </button>
                    )}
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
    <div className="mt-4 text-center">
      {voters.length >= 10 ? (
        <button
          aria-current="page"
          className="rounded-full px-6 py-2.5 bg-blue-600
          font-medium text-xs leading-tight
          uppercase hover:bg-blue-700 focus:bg-blue-700
          focus:outline-none focus:ring-0 active:bg-blue-800
```

```
            transition duration-150 ease-in-out dark:text-gray-300
            dark:border dark:border-gray-500 dark:bg-transparent"
          >
            Load More
          </button>
        ) : null}
      </div>
    </div>
  )
}

export default Voters
```

- Tạo folder “views” trong “src” với các file .jsx sau:

+ Home.jsx

```
import Banner from '../components/Banner'
import CreateProposal from '../components/CreateProposal'
import Proposals from '../components/Proposals'

const Home = () => {
  return (
    <>
      <Banner />
      <Proposals />
      <CreateProposal />
    </>
  )
}

export default Home
```

+ Proposal.jsx

```
import ProposalDetails from '../components/ProposalDetails'
import Voters from '../components/Voters'

const Proposal = () => {
  return (
    <>
```

```
      <ProposalDetails />
      <Voters />
    </>
  )
}

export default Proposal
```

- Tiếp theo sửa App.js => App.jsx và thêm code sau:

```
import { useEffect, useState } from 'react'
import { Routes, Route } from 'react-router-dom'
import { ToastContainer } from 'react-toastify'
import 'react-toastify/dist/ReactToastify.min.css'
import {
  getInfo,
  getProposals,
  isWallectConnected,
} from './Blockchain.services'
import Header from './components/Header'
import Home from './views/Home'
import Proposal from './views/Proposal'

const App = () => {
  const [loaded, setLoaded] = useState(false)
  useEffect(async () => {
    await isWallectConnected()
     getInfo()
     getProposals()
    setLoaded(true)
  }, [])

  return (
    <div className="min-h-screen bg-white text-gray-900 dark:bg-[#212936]
dark:text-gray-300">
      <Header />
      {loaded ? (
        <Routes>
          <Route path="/" element={<Home />} />
```

```
          <Route path="/proposal/:id" element={<Proposal />} />
        </Routes>
      ) : null}

      <ToastContainer
        position="top-center"
        autoClose={5000}
        hideProgressBar={false}
        newestOnTop={false}
        closeOnClick
        rtl={false}
        pauseOnFocusLoss
        draggable
        pauseOnHover
      />
    </div>
  )
}

export default App
```

- Sửa index.css thành code sau:

```
@import
url('https://fonts.googleapis.com/css2?family=Open+Sans:wght@300;400;500;
600;700&display=swap');

* html {
  padding: 0;
  margin: 0;
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: 'Open Sans', sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
```

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

- Sửa index.js => index.jsx và thêm code sau:

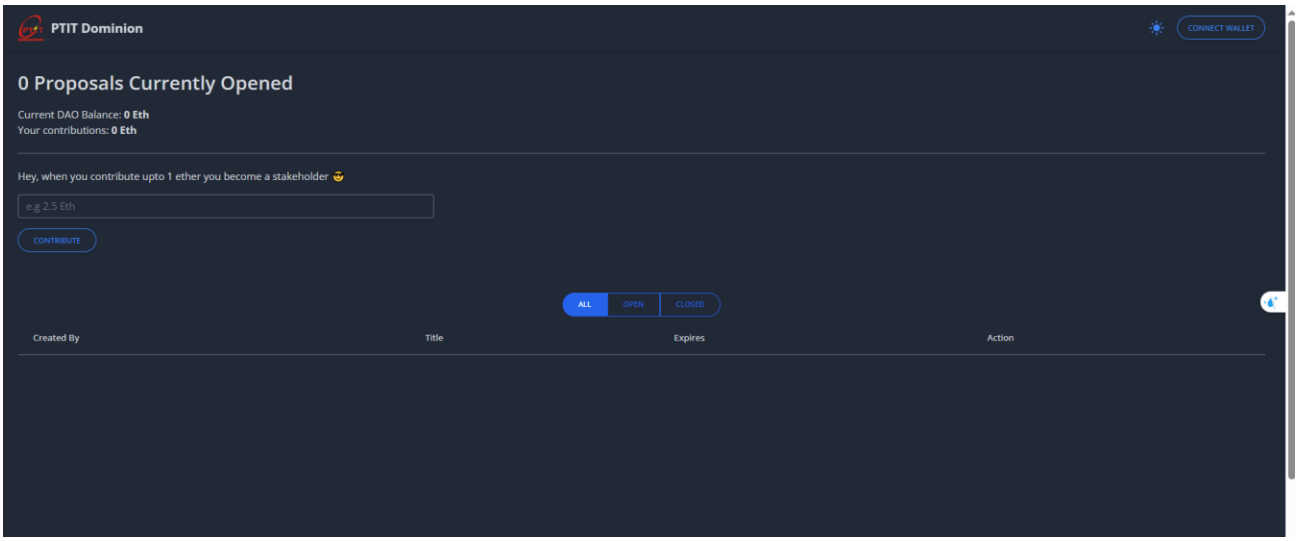
```
import React from 'react'
import ReactDOM from 'react-dom'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App'

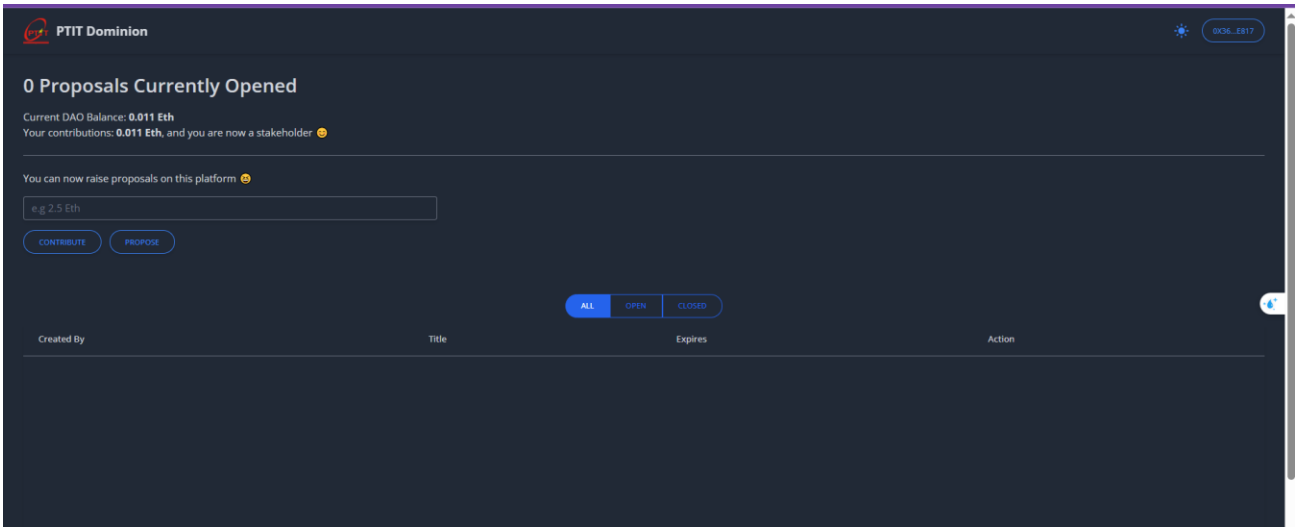
ReactDOM.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>,
  document.getElementById("root")
);
```

- Cuối cùng chạy câu lệnh để tạo web3 và truy cập htpp://localhost:3000/

- npm start

- Kết quả:





4. Thao tác với Web3

- Sử dụng chung 1 AddressContract để thực hiện thao tác

a. Kết nối ví (Connect Wallet)

- Trên giao diện web, bấm nút "Connect Wallet" ở góc trên bên phải.
- MetaMask sẽ mở popup yêu cầu bạn chọn tài khoản và nhấn "Connect".
- Sau khi kết nối thành công, địa chỉ ví rút gọn (VD: 0x1234...abcd) sẽ hiển thị thay cho nút Connect.

Lưu ý UX:

- Nếu chưa cài MetaMask → hiển thị thông báo "Vui lòng cài MetaMask".
- Nếu người dùng đổi mạng hoặc đổi tài khoản → UI sẽ tự reload trạng thái.

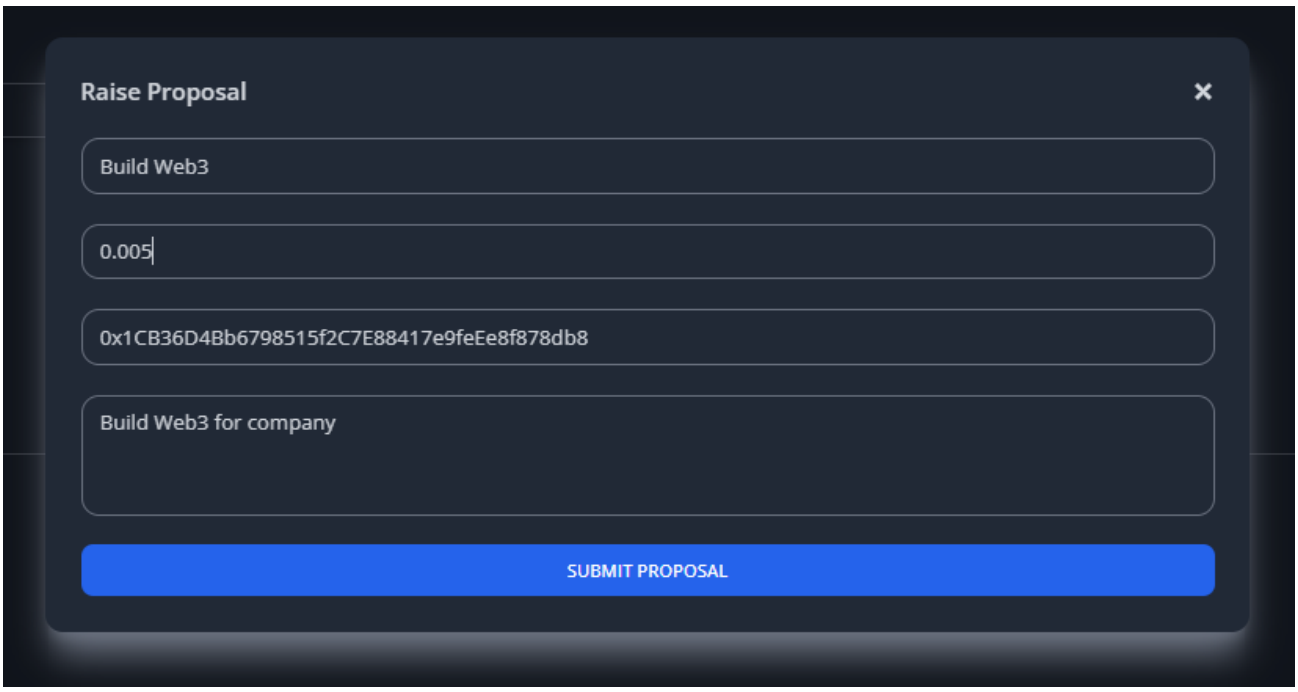
b. Đóng góp (Contribute)

- Ở màn hình chính, nhập số ETH muốn đóng góp (ví dụ 0.05).
- Bấm nút "Contribute" → MetaMask mở popup xác nhận giao dịch.
- Sau khi giao dịch thành công:
  - Số dư quỹ DAO (daoBalance) sẽ cập nhật.
  - Nếu tổng đóng góp ≥ 0.01 ETH, tài khoản trở thành Stakeholder.

c. Tạo đề xuất (Create Proposal)

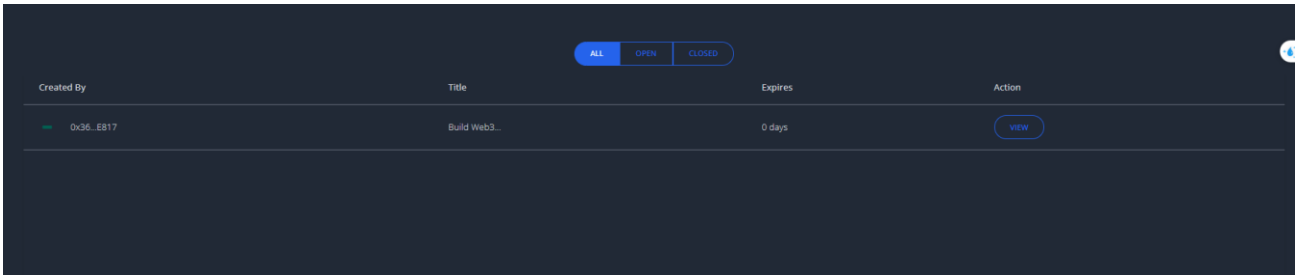
- Chỉ Stakeholder mới thấy nút "Propose".
- Khi click, mở modal với form:

    - Title: tiêu đề đề xuất.
    - Description: mô tả chi tiết.
    - Beneficiary: địa chỉ ví người nhận tiền.
    - Amount (ETH): số ETH muốn yêu cầu từ quỹ.
- Nhấn “Create Proposal” → MetaMask mở popup xác nhận.
- Sau khi tạo thành công, đề xuất mới hiển thị trong danh sách.



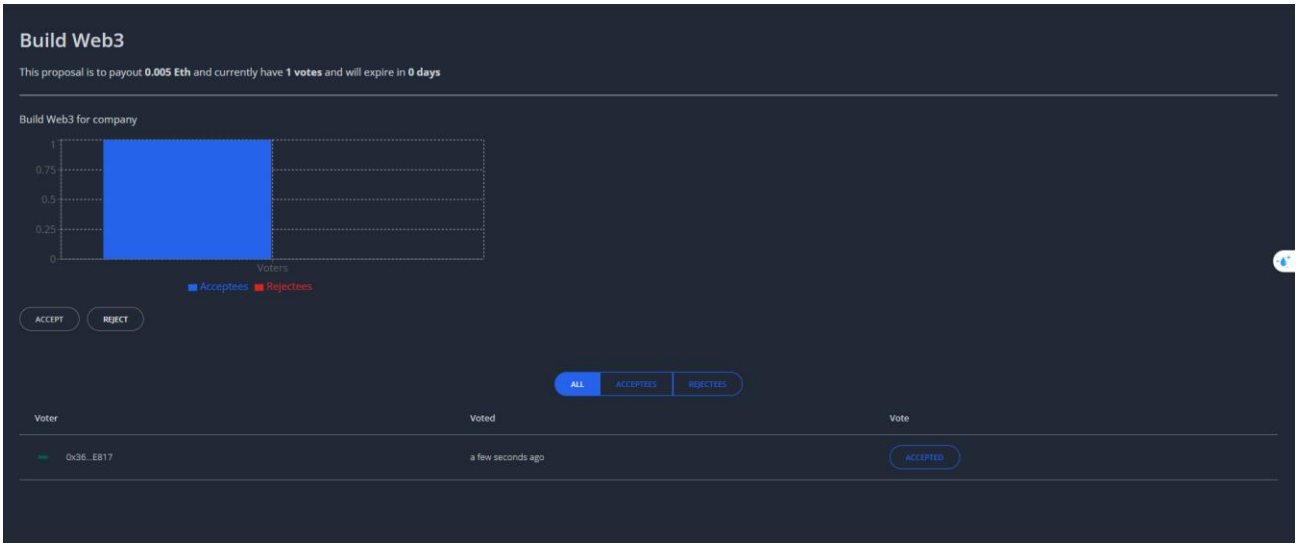
d. Xem danh sách đề xuất

- Trang chủ hiển thị tất cả đề xuất trong dạng cards hoặc bảng.
- Mỗi đề xuất gồm:
    - Tiêu đề + mô tả ngắn gọn
    - Số tiền yêu cầu
    - Người tạo đề xuất
    - Thời gian còn lại để vote
    - Số phiếu: 👍Upvotes / 👎 Downvotes
    - Trạng thái: Pending / Passed / Not Passed / Paid



e. Bỏ phiếu (Vote)

- Stakeholder thấy nút “Vote” trên mỗi đề xuất đang mở.
- Khi click: chọn “ACCEPT” hoặc “REJECT”.
- MetaMask mở popup xác nhận.
- Sau khi vote thành công, số phiếu cập nhật realtime.

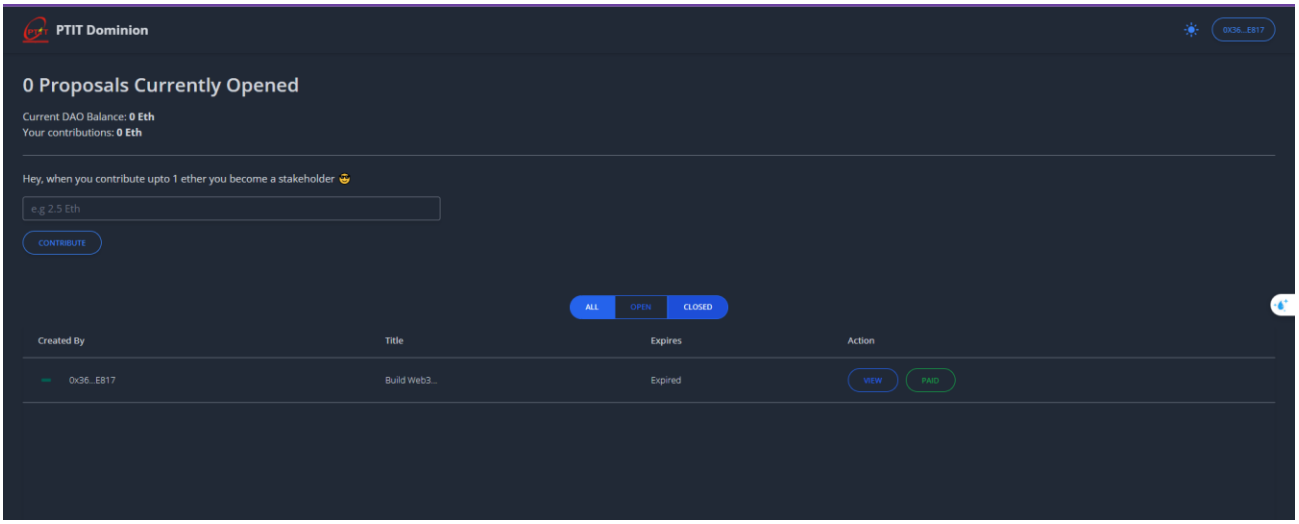


f. Thanh toán (Payout)

- Chỉ Stakeholder mới có quyền bấm “Payout”.
- Khi nhấn, MetaMask mở popup xác nhận.
- Sau khi thành công:
  - Tiền được chuyển đến beneficiary.
  - Trạng thái đề xuất → “Paid”.

👉 UI feedback: Hiển thị thông báo:

✅ “Payout thành công! 0.005 ETH đã gửi đến 0xABC...123”.


PTIT Dominion
0X36...E817
0 Proposals Currently Opened
Current DAO Balance: 0 Eth
Your contributions: 0 Eth
Hey, when you contribute upto 1 ether you become a stakeholder
e.g 2.5 Eth
CONTRIBUTE
ALL
OPEN
CLOSED
Created By
Title
Expires
Action
0x36...E817
Build Web3...
Expired
VIEW
PAID

Bài thực hành:

- Làm web3 theo hướng dẫn trên

- Một nhóm 3 đến 5 người sử dụng chung 1 address smartcontract và thao tác:

  - Đóng ETH để thành cổ đông
  - Tạo Vote bất kỳ cho địa chỉ một bạn A bất kỳ trong nhóm
  - Thực hiện vote
  - Sau 5 phút thực hiện thanh toán
  - Kiểm tra A đã nhận được tiền chưa

END!